

RAPPORT DE PROJET

IoT - Du capteur au serveur Web : Connecter, Sécuriser, Analyser



ANNÉE 2024-2025

Auteurs :

Senthalan THAYAPARAN
Sirius DOS SANTOS

Tuteur :

M. Cédric Brazeilles

Encadrant :

M. Rodrigue Michelin

Table des Matières

Introduction	02
Architecture globale du projet	03
Diagramme de cas d'utilisation	04
Matériels utilisés	05
Capteurs et communication ESP32	12
Architecture serveur - Raspberry Pi 5	21
Interface Web (Frontend PHP)	34
Sécurité de l'application IoT	36
Tests fonctionnels du système IoT	38
Difficultés rencontrées	43
Perspectives et Améliorations	44
Conclusion	46
Annexes	47

1 Introduction

L'Internet des Objets (IoT - Internet of Things) joue un rôle essentiel dans la collecte, la transmission et l'analyse des données en temps réel. Grâce aux objets connectés, il est possible d'optimiser divers domaines tels que l'industrie, la domotique, l'agriculture ou encore la santé, permettant ainsi une gestion plus efficace des ressources et une prise de décision plus rapide.

Cependant, plusieurs défis majeurs doivent être relevés pour garantir l'efficacité de ces systèmes : l'interopérabilité des dispositifs hétérogènes, la connectivité sur de longues distances, la gestion de l'énergie, et surtout la sécurisation des échanges de données.

Dans ce contexte, notre objectif principal a été de concevoir et de mettre en place une infrastructure IoT complète, allant de la collecte des données environnementales jusqu'à leur visualisation sécurisée sur une interface web. Nous avons également accordé une attention particulière à la sécurité des échanges de données tout au long de cette chaîne.

Concrètement, notre système s'appuie sur plusieurs capteurs connectés à des micro-contrôleurs ESP32, ainsi que sur des modules LoRa pour la communication longue portée. Les capteurs utilisés sont les suivants :

- **LM363** : Capteur de luminosité.
- **YL-69** : Capteur d'humidité du sol.
- **DHT22** : Capteur de température et d'humidité.

Les données mesurées sont transmises via des modules ESP32 (ESP-NOW ou Wi-Fi), ou à travers une communication LoRa (dans le cadre d'un déploiement longue portée). Elles sont ensuite stockées de manière sécurisée dans une base de données **PostgreSQL**, et accessibles via une **API REST** développée en Java avec le framework **Spring Boot**. Enfin, une interface web sécurisée en **PHP** permet à chaque utilisateur connecté de visualiser dynamiquement les données selon ses autorisations.

Ce rapport retrace l'ensemble du projet : les choix technologiques qui ont été faits, la mise en œuvre de l'architecture matérielle et logicielle, ainsi que les résultats obtenus. Nous y exposerons également les difficultés rencontrées, notamment dans la mise en place du réseau LoRa, ainsi que les améliorations possibles et les perspectives futures du système.



2 Architecture globale du projet

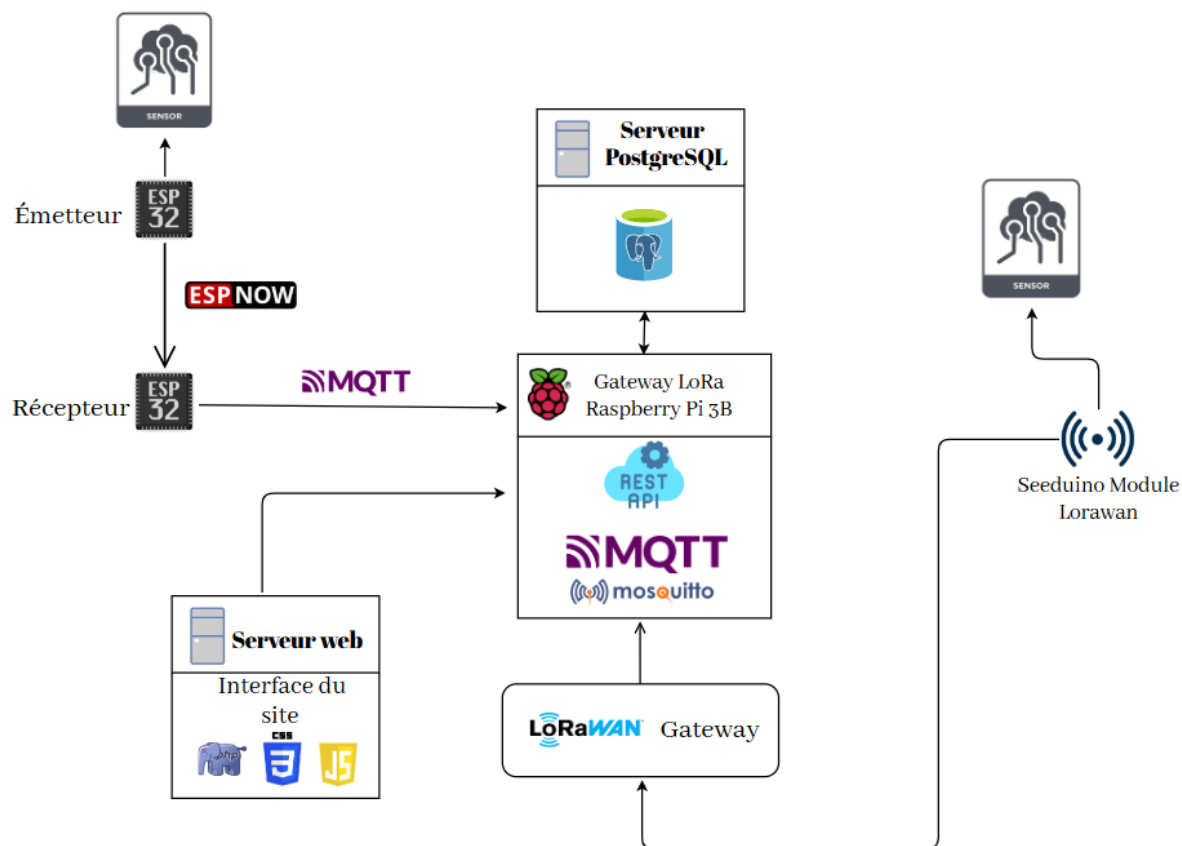


FIGURE 2.1 – Schéma de l'architecture globale du projet

Ce schéma représente l'architecture globale du projet IoT développé. Il présente clairement les flux de données depuis les capteurs jusqu'à leur visualisation sur l'interface web. Dans un premier temps, les capteurs environnementaux sont reliés à un microcontrôleur ESP32 émetteur qui collecte les données mesurées. Celui-ci transmet ensuite ces données via le protocole **ESP-NOW** vers un deuxième ESP32 récepteur, chargé de les transférer au serveur central en utilisant le protocole **MQTT**. Parallèlement, un module LoRaWAN (Seeeduino) envoie les données directement vers une gateway LoRaWAN, assurant une communication longue portée. Les données collectées par ces deux voies arrivent ensuite sur une passerelle Raspberry Pi 3B qui héberge à la fois un broker MQTT (Mosquitto) et une API REST permettant l'échange sécurisé de données avec une base de données **PostgreSQL**. Enfin, les données stockées sont rendues accessibles via une interface web sécurisée développée en PHP, HTML, CSS et JavaScript, permettant à l'utilisateur final une visualisation dynamique et sécurisée des données environnementales mesurées par les différents capteurs du réseau.

3 Diagramme de cas d'utilisation

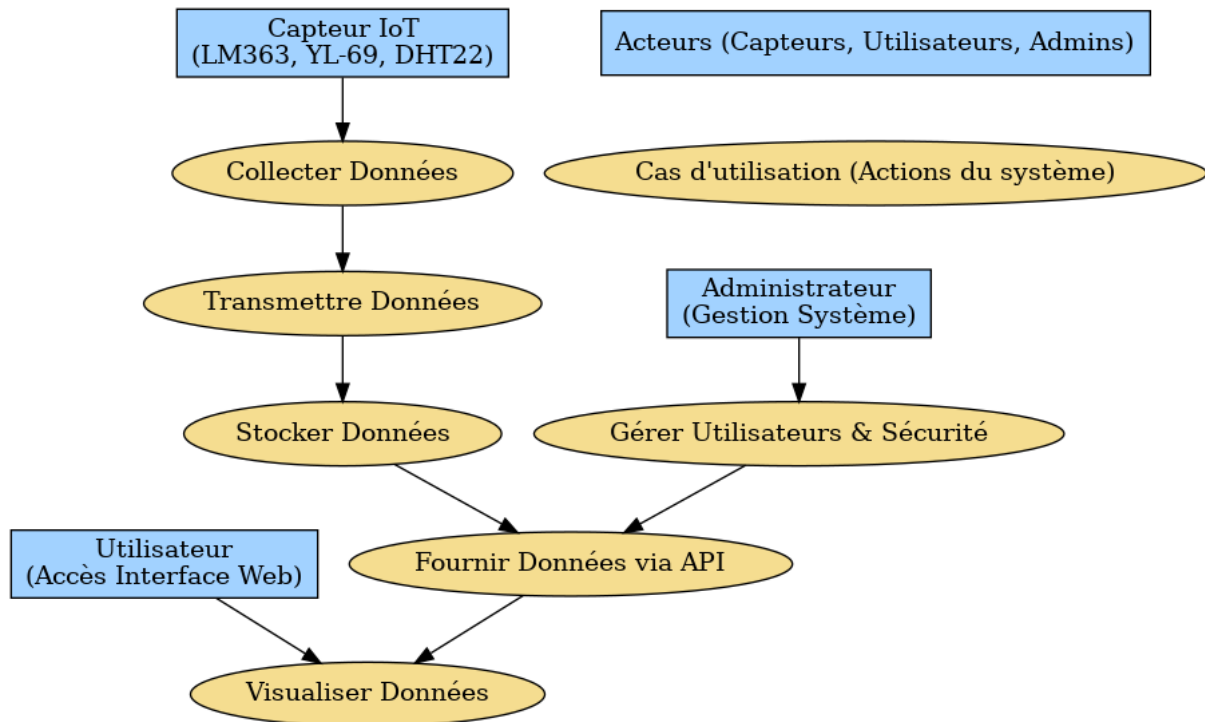


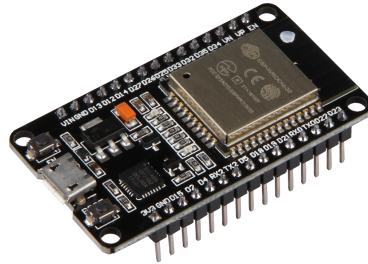
FIGURE 3.1 – matériel utmmme de cas d'utilisation

Ce matériel utmmme d'utilisation représente les interactions entre les acteurs et les fonctionnalités principales d'une infrastructure IoT complète, depuis la collecte des données jusqu'à leur visualisation sécurisée. Il présente trois acteurs principaux : les capteurs IoT (LM363, YL-69, DHT22) qui collectent les données environnementales, l'utilisateur final qui accède à l'interface web pour visualiser les informations selon ses autorisations, et l'administrateur du système qui gère les utilisateurs et garantit la sécurité des échanges. Les cas d'utilisation illustrés incluent la collecte des données par les capteurs, leur transmission via des modules ESP32 utilisant des communications LoRa ou Wi-Fi, le stockage sécurisé des données dans une base PostgreSQL, l'accès aux données via une API REST développée avec Spring Boot, et enfin la visualisation dynamique des données à travers une interface web sécurisée en PHP. Ce matériel utilisisme permet ainsi d'identifier clairement les responsabilités, interactions et flux d'informations entre les acteurs et les différentes fonctions du système IoT.

4 Matériels utilisés

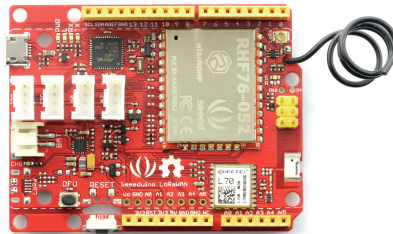
Dans le cadre de ce projet, nous avons utilisé les éléments matériels suivants pour la collecte de données, la communication sans fil, le traitement et la visualisation via une interface web sécurisée :

- **ESP32 (Émetteur/Récepteur)**



Microcontrôleur avec connectivité Wi-Fi et Bluetooth, utilisé ici pour récupérer les données des capteurs et les transmettre via ESP-NOW ou MQTT.

- **Seeeduno LoRaWAN RHF76-052DM**



Carte compatible Arduino intégrant un module LoRaWAN RHF76-052DM, utilisée pour tester la communication longue portée en LoRa point-à-point avec d'autres modules.

- **Module LoRa RHF0M301 (Concentrateur)**



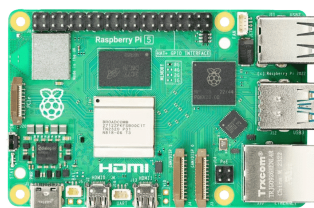
Module de communication LoRa utilisé dans la tentative de mise en place d'une gateway LoRa. Il fonctionne avec une interface SPI et est prévu pour recevoir les trames LoRa des capteurs.

- **Bridge SPI PRI-2 RHF4T002**



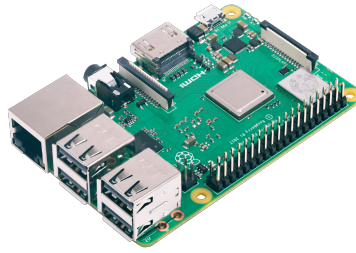
Bridge utilisé pour interfacier le module concentrateur RHF0M301 avec la Raspberry Pi via SPI. Il permet la communication entre le module LoRa concentrateur et le système embarqué (Pi 3B).

- **Raspberry Pi 5**



Serveur principal du projet, hébergeant l'API REST développée avec Spring Boot, le broker MQTT (Mosquitto), et le site web PHP de visualisation. Elle centralise toutes les données reçues.

- Raspberry Pi 3B (Gateway LoRa)



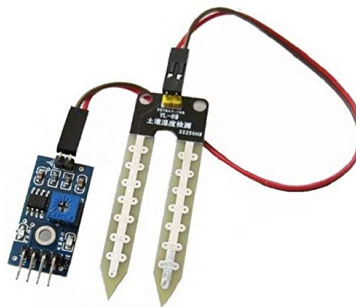
Utilisée pour héberger le module concentrateur LoRa RHF0M301 connecté via le bridge SPI. Elle était prévue comme passerelle LoRa pour réceptionner les trames des capteurs LoRaWAN.

- LM363 (Capteur de luminosité)



Capteur analogique permettant de mesurer l'intensité lumineuse ambiante.

- YL-69 (Capteur d'humidité du sol)



Capteur analogique mesurant le taux d'humidité dans le sol via la résistance entre deux sondes métalliques.

- **DHT22 (Capteur température et humidité)**



Capteur numérique mesurant la température ambiante et l'humidité relative avec précision.

- **Potentiomètre**



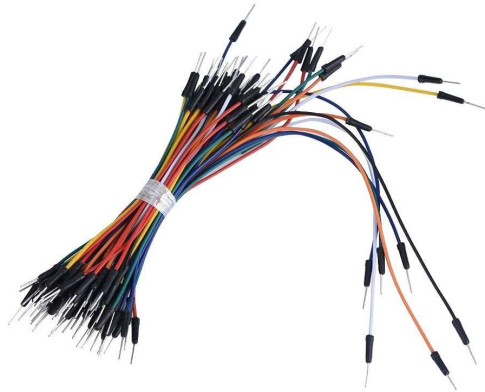
Composant passif utilisé pour ajuster la luminosité (contraste) de l'affichage LCD en modifiant la tension d'entrée du module LCD. En tournant sa molette, il permet d'augmenter ou diminuer le contraste pour une meilleure lisibilité de l'écran.

- **Câble Mini USB**



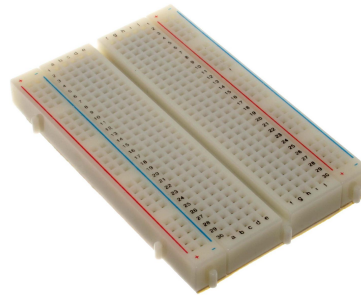
Permet de connecter les microcontrôleurs à l'ordinateur pour l'alimentation, la programmation ou le transfert de données.

- **Fils de connexion (Dupont)**



Utilisés pour les connexions entre les capteurs, les modules LoRa et les microcontrôleurs sans soudure.

- **Breadboard (Platine d'expérimentation)**



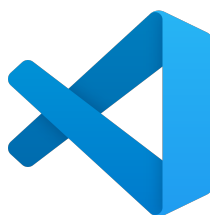
Support permettant de réaliser rapidement des montages électroniques temporaires sans soudure. Idéal pour tester, modifier et valider des circuits expérimentaux avant une intégration finale.

Logiciels et Systèmes d'Exploitation

Pour mener à bien ce projet, différents logiciels ont été utilisés en fonction du matériel et des rôles attribués à chaque composant du système. Les voici classés par plateforme.

Poste de développement (PC personnel)

- **Visual Studio Code**



Utilisé pour le développement en Python, HTML/CSS, configuration MQTT, fichiers ESP32, etc.

- **Arduino IDE**



Utilisé pour la programmation des ESP32, le test des capteurs et la communication ESP-NOW / LoRa.

- **IntelliJ IDEA**



Utilisé pour développer l'API REST en Java avec le framework Spring Boot.

- **Git**



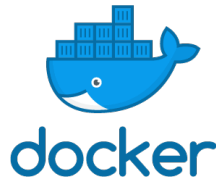
Outil de gestion de versions distribué, utilisé pour assurer le suivi des modifications du code source. Il nous a permis de faciliter la collaboration à distance lors du développement du site, ainsi que de maintenir l'historique complet des changements apportés aux fichiers du projet.

- **VirtualBox et Kali Linux**



Utilisé pour les tests de sécurité et l'analyse réseau, permettant de réaliser des audits de sécurité, du monitoring et des évaluations de vulnérabilités des systèmes du projet.

- Docker



Utilisé pour la conteneurisation des applications, facilitant le déploiement, la scalabilité et la gestion des environnements d'exécution, assurant ainsi une portabilité et une isolation optimales des services.

Raspberry Pi 5 (Serveur IoT)

- **Système d'exploitation** : Raspberry Pi OS Lite
- **Spring Boot** : Framework Java pour API REST
- **Mosquitto** : Broker MQTT
- **PHP et Chart.js** : Développement de l'interface web
- **Apache2** : Serveur HTTP

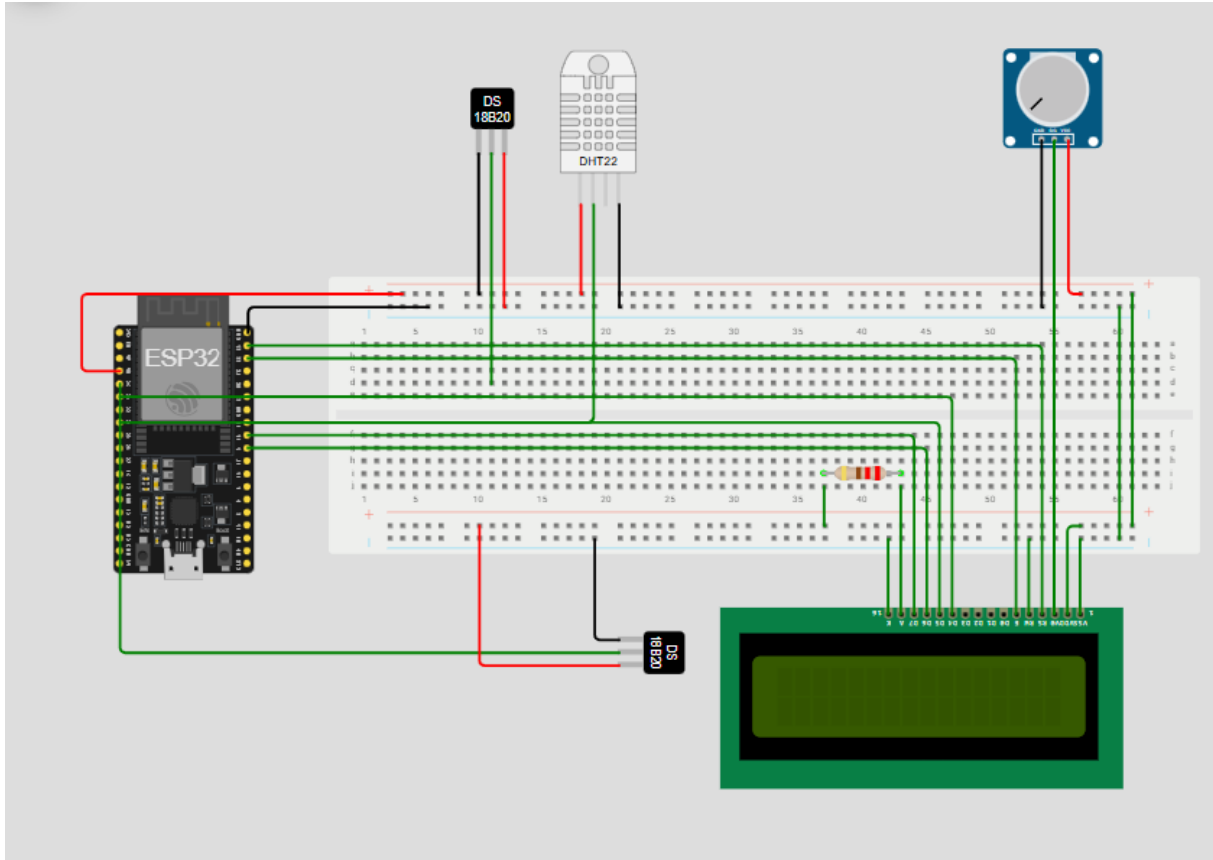
Raspberry Pi 3B (Passerelle LoRa)

- **Système d'exploitation** : Raspberry Pi OS Lite
- **Utilitaires SPI (pour RHF0M301)** : spi-tools, spidev, gpod, gpio, libgpod
- **Scripts Python pour tests LoRa** : spidev_test et Python pour tests SPI
- **Packet Forwarder** : Semtech Packet Forwarder, ChirpStack Gateway Bridge
- **Serveur ChirpStack** : Gestion des données LoRaWAN
- **Bibliothèques nécessaires** : libloragw, mbedtls (gestion TLS/SSL)
- **Gestion des GPIO** : libgpod, gpiset
- **Logiciels de test supplémentaires** : util_pkt_logger, util_rx_test
- **Outils de vérification réseau** : ping, netstat, ssh, scp
- **Alimentation et matériel** : Alimentation 5V/2.5A pour Raspberry Pi, alimentation LiPo pour RHF0M301 (optionnelle)

5 Capteurs et communication ESP32

Le système utilise deux microcontrôleurs ESP32 qui collaborent pour mesurer et transmettre des données environnementales vers un serveur distant. Ces deux rôles sont distincts : un ESP32 dédié à la collecte (émetteur) et l'autre à la réception et transmission vers le serveur MQTT (récepteur).

ESP32 Émetteur – Lecture des capteurs et transmission des données



Fonctionnement global du code :

- Initialisation des composants
 - Capteur DHT22 pour la température et l'humidité (GPIO 33).
 - LM363 pour la luminosité (GPIO 34).
 - YL-69 pour l'humidité du sol (GPIO 35).
 - Écran LCD 16x2 pour afficher les valeurs localement.

- **Configuration ESP NOW avec chiffrement**

- Le module ESP32 est configuré en mode station (`WiFi.mode(WIFI_STA)`).
- Le chiffrement ESP NOW est activé avec une clé de 16 octets définie dans le code (LMK).
- L'adresse MAC du récepteur est précisée et ajoutée avec la clé via la fonction `esp_now_add_peer()`.

- **Structure des données**

Une structure `struct_message` est utilisée pour regrouper les données envoyées : température, humidité, luminosité, humidité du sol et l'adresse MAC de l'émetteur.

- **Lecture des capteurs et envoi**

- Toutes les 2 secondes, les valeurs sont relevées.
- Les valeurs analogiques sont converties en pourcentages.
- La structure est remplie puis envoyée au récepteur via la fonction `esp_now_send()`.

- **Affichage sur l'écran LCD**

- Les quatre mesures principales (température, humidité, luminosité, humidité du sol) sont affichées en deux lignes.

Clé de chiffrement ESP-NOW : Le chiffrement est activé avec une clé partagée de 16 octets (`uint8_t lmk[16] = {...}`) que les deux microcontrôleurs doivent utiliser pour communiquer de manière sécurisée.

ESP-NOW – Communication chiffrée entre ESP32

ESP-NOW permet une communication directe et rapide entre ESP32, avec possibilité d'activer le chiffrement AES-128.

Avantages :

- Point-à-point, sans routeur Wi-Fi
- Très faible latence
- Faible consommation énergétique
- **Chiffrement AES possible** pour protéger les données sensibles

Dans notre projet : l'ESP32 **émetteur** envoie des données chiffrées à l'ESP32 **récepteur**, avec une clé secrète partagée.

ESP32 Récepteur – Traitement et transmission des données

L'ESP32 récepteur joue un rôle de passerelle entre l'émetteur (via ESP-NOW) et le serveur backend (via MQTT). Il reçoit les données environnementales envoyées par l'émetteur, les affiche sur un écran LCD, puis les transmet au broker MQTT Mosquitto hébergé sur la Raspberry Pi 5.

Fonctionnement global :

- Réception des données capteurs via ESP-NOW (sans passer par un point d'accès Wi-Fi).
- Affichage local des valeurs sur écran LCD 16x2.
- Publication des données au format JSON sur le topic MQTT `esp32/sensors`.
- Reconnexion automatique au broker en cas de perte de connexion.

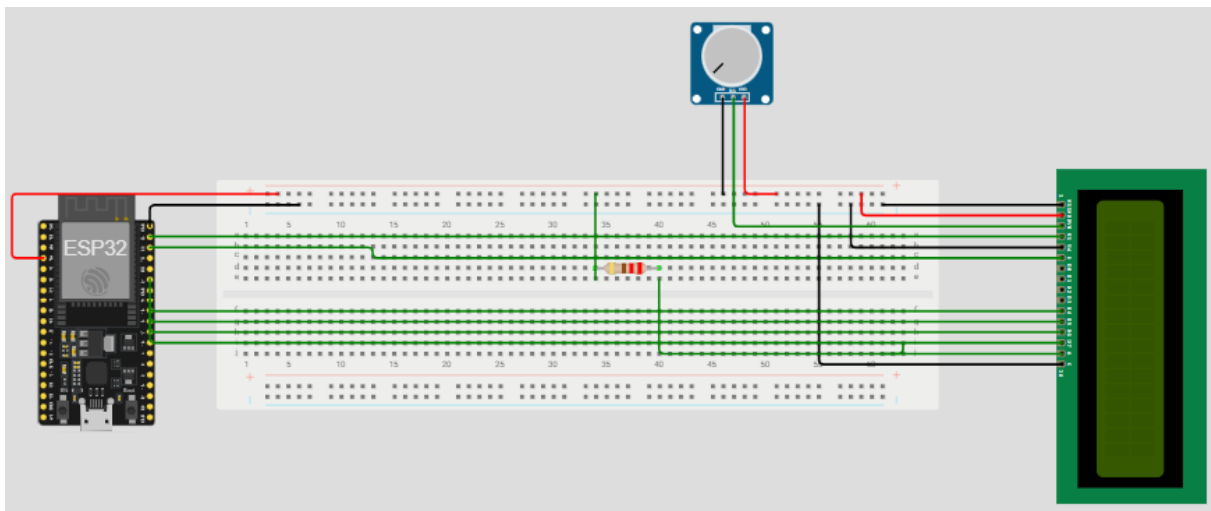


FIGURE 5.1 – Schéma de principe de l'ESP32 Récepteur et communication MQTT

1. Réception des données via ESP-NOW

Lorsqu'un paquet est reçu depuis l'émetteur, la fonction callback `OnDataRecv()` est automatiquement appelée. Elle extrait les données depuis la mémoire et les stocke dans une structure contenant :

- Température
- Humidité
- Luminosité brute
- Humidité du sol brute
- Adresse MAC de l'émetteur

Ces données sont immédiatement affichées sur l'écran LCD puis transmises au serveur MQTT.

2. Affichage local sur écran LCD

L’affichage est géré par la fonction `afficherLCD()` :

- Ligne 1 : Température (°C) et Humidité (
- Ligne 2 : Luminosité (

Les valeurs brutes sont converties en pourcentages grâce à la fonction `map()`, afin d’être plus lisibles.

3. Transmission via MQTT

Les données reçues sont converties en format JSON dans la fonction `sendToMQTT()` :

```
{  
  "temperature": 23.8,  
  "humidity": 51.6,  
  "luminosite_raw": 3220,  
  "humidite_sol_raw": 2150,  
  "macAddress": "AA:BB:CC:DD:EE:FF"  
}
```

Ces données sont publiées sur le topic MQTT `esp32/sensors`. Le serveur backend (Spring Boot) s’abonne à ce topic et les insère automatiquement dans la base de données PostgreSQL.

4. Connexion au réseau Wi-Fi

Avant toute opération, le récepteur se connecte au réseau local (point d’accès mobile, box, etc.) pour pouvoir communiquer avec le broker MQTT.

5. Gestion de la connexion MQTT

Dans la boucle principale (`loop()`), la fonction `reconnect()` permet de se reconnecter automatiquement au broker MQTT si la connexion a été perdue. Cela garantit un envoi fiable des données.

MQTT – Transmission des données vers le serveur

MQTT (Message Queuing Telemetry Transport) est un protocole léger de messagerie publish/subscribe.

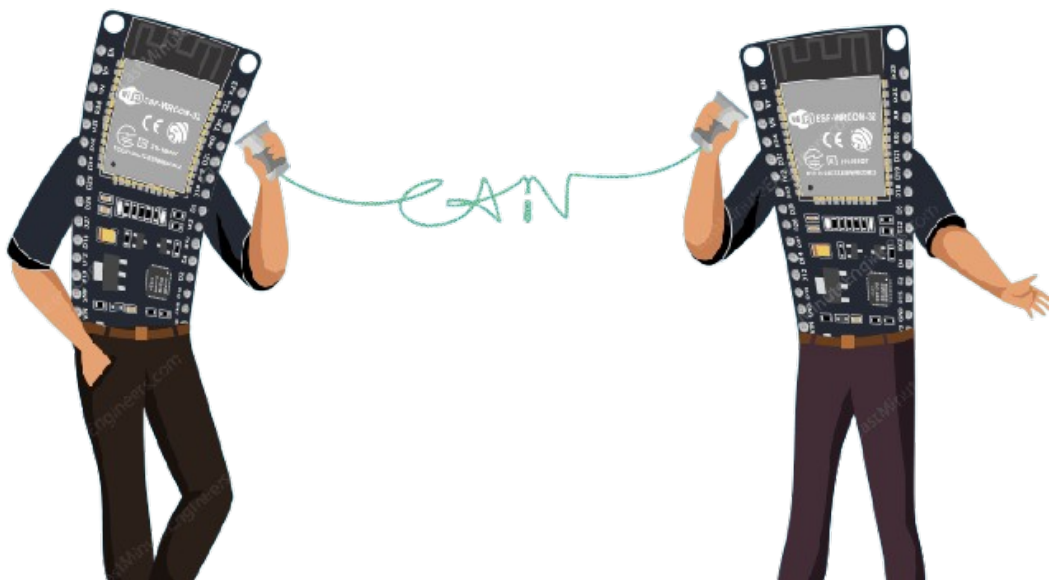
Avantages pour l'IoT :

- Faible bande passante utilisée
- Faible latence
- Communication en mode push
- Compatible avec les architectures événementielles

Dans notre projet :

- L'ESP32 récepteur publie les mesures sur le topic `esp32/sensors`.
- Le serveur Spring Boot écoute ce topic, puis enregistre les données en base de données PostgreSQL.

Grâce à cette architecture, le récepteur agit comme un pont entre les microcontrôleurs physiques et le serveur web distant, assurant la continuité de la collecte des données et leur visualisation en temps réel via le frontend PHP.



Tentative LoRaWAN et réorientation du projet

À l'origine, notre projet portait sur des thématiques liées à la cybersécurité. Cependant, cette approche ne s'est pas révélée suffisamment pertinente ni exploitable dans le cadre de notre échéance. Nous avons donc décidé de réorienter notre travail vers un projet IoT centré sur la collecte et la transmission de données environnementales à l'aide de capteurs connectés.

Notre objectif initial était d'intégrer une communication longue portée grâce à la technologie LoRaWAN. Malgré plusieurs tentatives techniques détaillées ci-après, cette partie du projet n'a pas pu être finalisée. Nous avons alors concentré nos efforts sur une architecture fonctionnelle à base d'ESP32 et du protocole ESP-NOW pour la transmission locale de données.

Nous avons utilisé un capteur **Seeeduino RHF76-052DM** capable d'envoyer des données via LoRa point-à-point. Ce dernier affichait bien les données sur son écran LCD, preuve que les trames étaient bien émises localement. Le principal défi a été la réception : malgré plusieurs essais, aucune trame n'a pu être captée sur la gateway LoRa que nous avons tenté de mettre en place avec une **Raspberry Pi 3B**.

Architecture matérielle prévue

- **Émetteur** : Seeeduino RHF76-052DM avec capteur DHT22 intégré.
- **Récepteur** : Raspberry Pi 3B + Module LoRa concentrateur RHF0M301 + Bridge SPI RHF4T002.

Une configuration SPI a été effectuée pour permettre la communication entre le module concentrateur et la Raspberry Pi. L'idée était de récupérer les trames LoRa et de les transmettre à ChirpStack ou à un packet forwarder personnalisé.

Deuxième tentative avec Dragino Shield

Une autre méthode a été testée en parallèle : l'utilisation d'un **Dragino LoRa Shield v1.4** connecté à une carte Arduino Uno, puis câblé manuellement sur les broches RX/TX pour surveiller les trames LoRa.

- Le shield a été alimenté correctement.
- RX/TX connectés à une autre carte pour observer le trafic.
- Aucun retour ni activité série détectée.
- Hypothèse : tension incompatible ou absence de signal LoRa au moment des tests.

Problèmes rencontrés

- Impossible de recevoir des trames côté Pi 3B.
- Tests SPI réussis avec `spidev`, mais le module concentrateur ne répondait pas.
- Doute sur le câblage du bridge RHF4T002 (niveau logique, reset, alimentation).
- Problème possible avec la version des outils `libloragw` ou la configuration GPIO (`gpioset`).
- Aucun log LoRa ni trame détectée avec `util_pkt_logger` ou `util_rx_test` (valeurs renvoyées à 0 en continu).
- Lors de l'exécution du packet forwarder, un message d'erreur générique `main error` apparaissait systématiquement.
- Aucun signal capté non plus avec le montage Dragino/Arduino.

Tests effectués sur la Raspberry Pi 3B

Afin d'identifier la source du dysfonctionnement, plusieurs commandes ont été exécutées en ligne de commande. Chaque test correspond à une capture d'écran fournie :

1. `ls /dev/spi*`

Vérifie que les interfaces SPI sont bien activées et visibles dans le système.
Résultat : `/dev/spidev0.0` et `/dev/spidev0.1` détectés.



```
admin@gateway:~ $ ls /dev/spi*
/dev/spidev0.0  /dev/spidev0.1
```

Qu'est-ce que le SPI ?

Le **SPI** (*Serial Peripheral Interface*) est un protocole de communication série synchrone utilisé pour échanger des données entre un microcontrôleur (comme une Raspberry Pi) et des périphériques externes (comme un module concentrateur LoRa).

Caractéristiques :

- Communication rapide (plus rapide que I2C dans la plupart des cas).
- Utilise 4 fils : MISO, MOSI, SCLK, CS.
- Le maître (ici la Raspberry Pi) contrôle la communication.

Dans notre projet : le SPI est utilisé pour permettre à la Raspberry Pi de communiquer avec le module concentrateur LoRa RHF0M301. Il est essentiel de bien configurer les broches et l'alimentation pour que l'échange fonctionne.

2. `lsmod | grep spi`

Permet de s'assurer que le module SPI est bien chargé dans le noyau.

```
admin@gateway:~$ lsmod | grep spi
spidev                16384  0
spi_bcm2835           20480  0
```

3. `dmesg | grep spi`

Affiche les logs système SPI. Utile pour repérer d'éventuels problèmes au démarrage.

Aucun message n'est retourné par la commande, ce qui peut signifier soit que le bus SPI est actif mais sans périphérique détecté, soit que la communication SPI a échoué.

4. `./util_pkt_logger`

Tentative de journalisation des trames. Résultat : vide.

```
admin@gateway:~/lora_gateway$ ./util_pkt_logger/util_pkt_logger
./util_spi_stress/util_spi_stress
loragw_pkt_logger: ERROR: failed to find any configuration file named global_conf.json, local_conf.json or debug_conf.js
on
INFO: Starting LoRa concentrator SPI stress-test number 1
Cycle 0 > error during the 1th iteration: write 0x67, read 0x00
Repeat read of target register: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
```

5. Packet Forwarder – main error

Le packet forwarder retourne une erreur fatale au lancement.

```
Version: 4.0.1
*** Lora concentrator HAL library version info ***
Version: 5.0.1;
***
INFO: Little endian host
INFO: found global configuration file global_conf.json, parsing it
INFO: global_conf.json does contain a JSON object named SX1301_conf, parsing SX1301 parameters
INFO: lorawan_public 1, clksrc 1
INFO: LBT is disabled
INFO: antenna_gain 0 dBi
INFO: Configuring TX LUT with 16 indexes
INFO: radio 0 enabled (type SX1257), center frequency 867500000, RSSI offset -166.000000, tx enabled 1, tx_notch_freq 129000
INFO: radio 1 enabled (type SX1257), center frequency 868100000, RSSI offset -166.000000, tx enabled 0, tx_notch_freq 0
INFO: Lora multi-SF channel 0> radio 1, IF -400000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 1> radio 1, IF -200000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 2> radio 1, IF 0 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 3> radio 0, IF -400000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 4> radio 0, IF -200000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 5> radio 0, IF 0 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 6> radio 0, IF 200000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora multi-SF channel 7> radio 0, IF 400000 Hz, 125 kHz bw, SF 7 to 12
INFO: Lora std channel> radio 1, IF -400000 Hz, 125000 Hz bw, SF 12
INFO: FSK channel> radio 1, IF 300000 Hz, 125000 Hz bw, 50000 bps datarate
INFO: global_conf.json does contain a JSON object named gateway_conf, parsing gateway parameters
INFO: gateway MAC address is configured to AA555A0000000000
INFO: server hostname or IP address is configured to "localhost"
INFO: upstream port is configured to "1680"
INFO: downstream port is configured to "1680"
INFO: downstream keep-alive interval is configured to 10 seconds
INFO: statistics display interval is configured to 30 seconds
INFO: upstream PUSH_DATA time-out is configured to 100 ms
INFO: packets received with a valid CRC will be forwarded
INFO: packets received with a CRC error will NOT be forwarded
INFO: packets received with no CRC will NOT be forwarded
INFO: found local configuration file local_conf.json, parsing it
INFO: redefined parameters will overwrite global parameters
INFO: local_conf.json does not contain a JSON object named SX1301_conf
INFO: local_conf.json does contain a JSON object named gateway_conf, parsing gateway parameters
INFO: gateway MAC address is configured to AA555A0000000000
INFO: packets received with a valid CRC will be forwarded
INFO: packets received with a CRC error will NOT be forwarded
INFO: packets received with no CRC will NOT be forwarded
ERROR: Failed to load fw 1
ERROR: Version of calibration firmware not expected, actual:0 expected:2
ERROR: [main] failed to start the concentrator
```

Conclusion de la tentative

Malgré plusieurs essais, la mise en place d'une passerelle LoRa complète n'a pas abouti. Les trames étaient bien envoyées mais jamais reçues par les concentrateurs testés. Cela s'explique par un manque de documentation claire sur les modules.

Par ailleurs, une tentative de test avec un module LoRa RN2483 branché en UART sur une Raspberry Pi n'a pas non plus abouti : bien que les commandes AT aient pu être envoyées, il s'est révélé incompatible avec l'architecture concentrateur requise pour capturer les trames LoRa d'un autre module.

Néanmoins, cette tentative a enrichi notre compréhension des communications longue portée, des interfaces SPI avancées, et des logiciels tels que **ChirpStack**.

Nous conservons cette architecture comme piste d'évolution pour un projet futur, avec plus de temps pour la mise en service complète. Le code utilisé sur le Seeeduino sera fourni en annexe.



6 Architecture serveur Raspberry Pi 5

Rôle de la Raspberry Pi 5 dans l'infrastructure

La Raspberry Pi 5 joue un rôle central dans l'architecture du projet. Elle agit comme serveur principal et concentre toutes les fonctions critiques suivantes :

- Réception des données via le broker MQTT Mosquitto.
- Traitement et stockage des mesures environnementales dans une base PostgreSQL.
- Exposition d'une API REST sécurisée (Spring Boot).
- Affichage des mesures via une interface web en PHP + Chart.js.

Ce choix s'inscrit dans une volonté d'évolution : bien que toutes les fonctions aient d'abord été centralisées sur la Raspberry Pi 5 pour des raisons de simplicité et de tests, l'objectif à terme est de déporter la base de données vers une machine dédiée, ce qui a déjà été prévu dans la configuration du backend.

Organisation logicielle :

L'ensemble du système logiciel repose sur une architecture en couches basée sur le **patron de conception MVC (Modèle - Vue - Contrôleur)** :

- Le **Modèle** correspond aux entités Java (**User**, **Device**, **Measurement**, etc.), gérées par l'ORM Hibernate et synchronisées avec la base PostgreSQL.
- Le **Contrôleur** correspond aux routes REST exposées via Spring Boot. Ces routes gèrent les échanges entre les utilisateurs, les capteurs et les mesures.
- La **Vue** est représentée par l'interface PHP, qui consomme l'API REST, interprète les données JSON et les affiche à l'aide de Chart.js.

Cette organisation rend le projet plus modulaire, lisible et évolutif. Chaque couche peut être modifiée ou remplacée indépendamment des autres, facilitant ainsi la maintenance et les évolutions futures (par exemple, remplacement du frontend PHP par une interface mobile).

Outils utilisés pour le développement collaboratif :

- **Maven** : Le projet backend utilise **Maven** comme outil de gestion de projet. Maven permet de déclarer toutes les dépendances nécessaires dans un seul fichier (**pom.xml**), de compiler le projet, de le packager sous forme de **.jar**, et de simplifier les mises à jour. C'est un outil central pour tout projet Java structuré.
- **Git / GitHub** : Le projet est versionné à l'aide de **Git**, avec un dépôt distant hébergé sur GitHub. Cela nous permet de travailler à deux de manière collaborative, de suivre les modifications, de revenir à des versions antérieures, et de créer des branches pour tester des fonctionnalités ou corriger des bugs.
- **Accessibilité du projet** :

Le dépôt Git du projet est **public** et contient l'intégralité du code source, y compris :

- le backend Spring Boot avec les entités, contrôleurs et services,
- l'interface web en PHP,
- les scripts pour les ESP32,
- les fichiers de configuration (Mosquitto, PostgreSQL, JWT...),

- une image Docker pour test de ntnt sur le cloud.

Le projet est disponible à l'adresse suivante :

https://github.com/collapsedbanana/API_IOT

Base de données – PostgreSQL gérée automatiquement par Hibernate

Dans notre architecture, la base de données PostgreSQL est utilisée pour stocker les données issues des capteurs ESP32, transmises via MQTT. Cette base n'est pas créée manuellement mais générée automatiquement grâce au framework Java Spring Boot et à sa couche ORM : **Hibernate**.

Qu'est-ce qu'un ORM ?

Un **ORM** (Object-Relational Mapping) permet de manipuler une base SQL à travers des objets Java. Les classes sont directement liées à des tables, et les relations (**OneToMany**, **ManyToOne**, etc.) décrivent la structure de la base.

Dans notre projet :

- L'ORM utilisé est **Hibernate**, intégré à Spring Boot.
- Chaque entité Java est synchronisée automatiquement avec la base PostgreSQL.
- La création, modification et suppression de lignes est gérée via le code Java, sans requête SQL explicite.

Avantages :

- Plus besoin d'écrire manuellement des requêtes **CREATE TABLE**.
- Maintenance facilitée : les changements de structure se font directement dans le code.
- Cohérence garantie entre la base de données et les entités métiers.

Fonctionnement dans notre projet :

- À chaque lancement de l'application Spring Boot, Hibernate lit les entités annotées avec **@Entity** et met à jour la base PostgreSQL.
- La configuration de la base (adresse IP, utilisateur, mot de passe) est définie dans le fichier **application.properties**.
- La directive **spring.jpa.hibernate.ddl-auto=update** permet d'adapter la structure sans intervention manuelle.

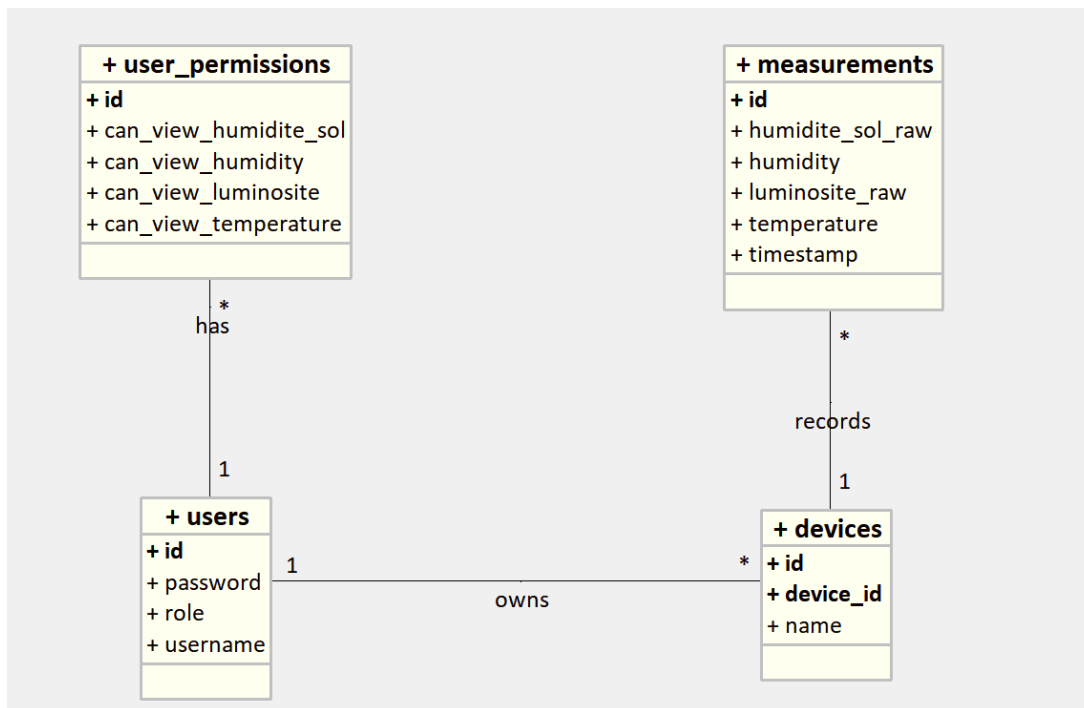


FIGURE 6.1 – matériel utilise UML généré automatiquement via les entités Hibernate

Structure des entités principales :

- `User.java` : représente un utilisateur de l'interface, avec son identifiant, nom, mot de passe hashé, rôle et ses droits.
- `Device.java` : correspond à un capteur physique (identifié par son adresse MAC). Il peut être lié à un utilisateur.
- `Measurement.java` : contient les données mesurées (température, humidité, luminosité, humidité du sol) pour chaque capteur.
- `UserPermission.java` : permet d'attribuer à des utilisateurs des droits d'accès sur des capteurs qu'ils ne possèdent pas directement.

Chaque entité Java est automatiquement convertie en une table SQL avec ses relations (clés étrangères, contraintes d'unicité, etc.). Grâce à cela, la base de données est pleinement synchronisée avec le code backend, réduisant le risque d'erreurs.

Les données envoyées par les capteurs ESP32 sont réceptionnées par le backend (via MQTT), puis stockées automatiquement dans la base grâce aux méthodes JPA de Spring Data. Il n'est donc pas nécessaire d'écrire du SQL pour insérer ou manipuler les données — cela est géré par l'ORM.

Installation de Mosquitto - Broker MQTT

Mosquitto est installé comme broker MQTT afin de recevoir les trames envoyées par l'ESP32 récepteur.

- Installation : `sudo apt install mosquitto mosquitto-clients`
- Configuration par défaut : port 1883 (non sécurisé).
- Topic utilisé : `esp32/sensors`
- Utilisation d'un client de test (`mosquitto_sub`) pour observer les trames.

Configuration du broker Mosquitto :

- Fichier principal : `/etc/mosquitto/mosquitto.conf`

```
admin@server:~ $ cat /etc/mosquitto/mosquitto.conf
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example

pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto/

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d
```

FIGURE 6.2 – Contenu du fichier mosquitto.conf

- Répertoire de configuration complémentaire : `/etc/mosquitto/conf.d/`

```
admin@server:~ $ ls /etc/mosquitto/conf.d/
cat /etc/mosquitto/conf.d/<ton_fichier.conf>
auth.conf  README
```

FIGURE 6.3 – Fichier auth.conf détecté dans conf.d

- Authentification activée avec `passwd` et droits définis par ACL.

```
admin@server:~ $ ls -l /etc/mosquitto/
ls -l /etc/mosquitto/certs/
cat /etc/mosquitto/passwd
total 32
-rw-r--r-- 1 root root 230 Jun 9 2021 aclfile.example
drwxr-xr-x 2 root root 4096 Mar 6 16:56 ca_certificates
drwxr-xr-x 2 root root 4096 Mar 6 16:56 certs
drwxr-xr-x 2 root root 4096 Mar 10 10:22 conf.d
-rw-r--r-- 1 root root 354 Sep 30 2023 mosquitto.conf
-rw-r--r-- 1 root root 123 Mar 19 00:59 passwd
-rw-r--r-- 1 root root 23 Jun 9 2021 pskfile.example
-rw-r--r-- 1 root root 355 Jun 9 2021 pwfile.example
total 4
-rw-r--r-- 1 root root 130 Sep 30 2023 README
test_user:$7$10!$pE0gZmGsLafnMHUC$h0Xa2+u57GzLYARfc0QQFe9micwKhFc8g+ZVXn1iVe6n0qgSGxDPiKi1D3qLy6ozwr2EDb0RCnSN2qo7yLIUg
==
```

FIGURE 6.4 – Mot de passe sécurisé pour l'utilisateur `test_user` et structure de Mosquitto

Note : Bien qu'un système SSL et une gestion fine des accès aient été mis en place via les fichiers `auth.conf`, nous avons temporairement désactivé cette configuration lors des tests avec les capteurs, en raison de certains problèmes de compatibilité ou de surcharge logicielle. Ce mécanisme reste cependant disponible et prêt à être réactivé.

Déploiement du backend Spring Boot – Détails techniques

Le backend repose sur l'utilisation du framework **Spring Boot** avec Java 17, comme dit précédemment. Il est conçu pour :

- S'abonner au topic `esp32/sensors` via MQTT.
- Insérer automatiquement les mesures dans la base PostgreSQL.
- Gérer l'authentification et les autorisations utilisateur via JWT.
- Exposer une API REST sécurisée permettant l'accès aux mesures.
- Communiquer uniquement en HTTPS via le port 8443.

Architecture logicielle et dépendances Maven

Le projet utilise Maven pour la gestion des dépendances. Voici les bibliothèques principales définies dans le fichier `pom.xml` :

Dépendances principales du backend

- `spring-boot-starter-web` : expose les routes REST.
- `spring-boot-starter-data-jpa` : communication avec PostgreSQL.
- `org.postgresql:postgresql` : pilote JDBC PostgreSQL.
- `org.eclipse.paho.client.mqttv3` : réception des messages MQTT.
- `jjwt` : gestion de l'authentification JWT (API, implémentation, Jackson).
- `lombok` : génération automatique du code (getters/setters).
- `spring-boot-starter-security` : sécurisation des endpoints.
- `android-json` : parsing JSON côté backend.
- `actuator` : monitoring de l'application.

Configuration détaillée du fichier application.properties

Le fichier application.properties contient l'ensemble des réglages essentiels du backend :

- **HTTPS activé sur le port 8443** : utilisation d'un keystore PKCS12.
- **Connexion à PostgreSQL** : URL + identifiants définis manuellement.
- **Topic MQTT** : esp32/sensors, via le broker Mosquitto.
- **Authentification JWT** : clé secrète définie manuellement.
- **Spring JPA** : création automatique des tables et schéma au lancement.

```
1  # =====
2  # Configuration HTTPS (8443)
3  # =====
4  server.address=0.0.0.0
5  server.port=8443
6  server.ssl.enabled=true
7  server.ssl.key-store=classpath:keystore.p12
8  server.ssl.key-store-password=123456
9  server.ssl.key-store-type=PKCS12
10 server.ssl.key-alias=tomcat
11
12 # =====
13 # Configuration PostgreSQL
14 # =====
15 spring.datasource.url=jdbc:postgresql://192.168.11.70:5432/mqtt_data?sslmode=disable&currentSchema=public
16 spring.datasource.username=esp32_user
17 spring.datasource.password=root
18 spring.datasource.driver-class-name=org.postgresql.Driver
19 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
20
21 # =====
22 # JPA / Hibernate
23 # =====
24 spring.jpa.hibernate.ddl-auto=update
25 spring.jpa.show-sql=true
26 spring.jpa.properties.hibernate.format_sql=true
27 spring.sql.init.mode=always
28
29 # =====
30 # MQTT Eclipse Paho
31 # =====
32 mqtt.broker=tcp://192.168.11.70:1883
33 mqtt.topic=esp32/sensors
34 mqtt.clientId=SpringBootMqttClient
35 mqtt.username=test_user
36 mqtt.password=test1234
37
38 # =====
39 # JWT
40 # =====
41 jwt.secret=E2ykpH0YQ9Ia0R49+x0R5qlERV0tnyQIGMJT4ilAucvrDGg//x7A3QqVqEqusv8c58QVvKzXk/76xSPnAH/s7A==
42
43 # =====
44 # Actuator
45 # =====
46 management.endpoints.web.exposure.include=*
```

FIGURE 6.5 – Configuration complète dans le fichier application.properties

Traitement des messages MQTT

Le backend implémente un service qui utilise le client Eclipse Paho pour s'abonner au topic `esp32/sensors`. Lorsqu'un message est reçu :

1. Il est parsé depuis le format JSON.
2. La valeur `macAddress` est utilisée pour retrouver le capteur correspondant.
3. Une nouvelle entité `Measurement` est créée avec les données reçues.
4. L'enregistrement est persisté via JPA dans la base PostgreSQL.

Authentification et sécurité JWT

Toutes les requêtes vers l'API sont protégées par un filtre `JwtAuthFilter`. Les utilisateurs doivent s'authentifier via une route spécifique (`/api/auth/login`) pour recevoir un token JWT.

Ce jeton est ensuite utilisé dans l'en-tête `HTTP Authorization: Bearer <token>` pour accéder aux autres routes.

Structure des entités (Java)

Chaque entité (`User`, `Device`, `Measurement`, `UserPermission`) est mappée vers une table PostgreSQL avec annotations JPA (`@Entity`, `@OneToMany`, etc.).

Les relations entre entités sont gérées automatiquement par Hibernate :

- Un utilisateur peut posséder plusieurs capteurs.
- Un capteur envoie plusieurs mesures.
- Des permissions peuvent être attribuées à chaque couple utilisateur-capteur.

Commandes de lancement

Le backend est compilé puis exécuté via les commandes suivantes sur la Raspberry Pi :

```
cd /home/pi/backend-api
compilation d'un snap-shot avec : mvn clean install
lancement : java -jar app.jar ou mvn spring-boot:run
```

Le processus tourne en arrière-plan et commence à écouter les messages MQTT automatiquement.

Cette couche logicielle assure la liaison entre les données physiques récoltées par les capteurs ESP32 et l'interface web, tout en assurant la sécurité, la cohérence des données, et une évolutivité forte du projet.

Les contrôleurs de l'application (Controller)

Comme mentionné précédemment, notre projet suit le patron de conception **MVC** (Modèle - Vue - Contrôleur). Après avoir présenté les modèles qui représentent nos entités persistées dans la base PostgreSQL, nous abordons désormais la couche **Controller**, qui gère les routes REST exposées à travers l'API Spring Boot.

Chaque contrôleur a un rôle spécifique et se concentre sur une entité ou un aspect fonctionnel précis de l'application.

- **AdminUserController.java** : Gère les opérations administratives liées aux utilisateurs.
 - Création d'un utilisateur (avec rôle et permissions par défaut).
 - Modification des permissions d'un utilisateur.
 - Suppression d'un utilisateur.
 - Récupération de la liste complète des utilisateurs (admin uniquement).
- **AuthController.java** : Gère l'authentification des utilisateurs.
 - Enregistrement d'un nouvel utilisateur.
 - Connexion avec vérification du mot de passe.
 - Génération d'un token JWT à la connexion.
 - Récupération des informations de l'utilisateur connecté.
- **MeasurementController.java** : Gère l'accès aux données mesurées par les capteurs.
 - Récupération des mesures associées à un capteur.
 - Récupération des mesures d'un utilisateur connecté, selon ses permissions.
 - Ajout manuel de mesures (utile pour les tests).
 - Suppression d'une mesure.
 - Récupération de toutes les mesures (admin).

Rôle des contrôleurs Spring Boot

Les contrôleurs reçoivent les requêtes HTTP entrantes et y répondent via des routes REST. Ils font le lien entre :

- Le client frontend (interface PHP ou outil API comme Postman)
- Le service (logique métier)
- Les entités de la base (via les repositories)

Chaque route est associée à un rôle d'utilisateur grâce aux annotations `@PreAuthorize`, garantissant la sécurité des accès selon les droits.

Les contrôleurs sont donc un point d'entrée essentiel dans l'architecture logicielle. Ils permettent une gestion fine des opérations et garantissent que seuls les utilisateurs autorisés accèdent aux données sensibles ou critiques. **Néanmoins, la logique métier complète (vérifications, traitements, gestion des droits...) est généralement déléguée aux services, décrits dans la prochaine section.**

Note : Le code source complet des contrôleurs (et des autres classes) est disponible en annexe, avec des commentaires intégrés pour faciliter la compréhension et la réutilisation du projet.

Accès aux données : les Repositories

Dans Spring Boot, les **repositories** permettent de manipuler les données en base de manière simplifiée. Ce sont des interfaces qui héritent de `JpaRepository` ou `CrudRepository`, et qui offrent automatiquement les méthodes de base comme `findAll()`, `save()`, `deleteById()`, etc.

Exemple de repository

```
public interface UserRepository extends JpaRepository<User, Long> {  
    User findByUsername(String username);  
}
```

Cela permet de rechercher un utilisateur à partir de son nom sans écrire de requête SQL.

Avantages :

- Réduction du code : les opérations CRUD sont prêtes à l'emploi.
- Intégration facile avec Hibernate pour interagir avec la base PostgreSQL.
- Possibilité de créer des méthodes personnalisées simplement avec des noms explicites.

Les repositories sont ainsi une passerelle directe entre la couche service/contrôleur et la base de données, tout en maintenant une grande lisibilité dans le code.

Les services – Logique métier

Dans l'architecture Spring Boot, la couche **Service** contient la logique métier centrale. Les contrôleurs REST délèguent leurs opérations aux services afin de séparer les responsabilités, faciliter la maintenance et permettre une meilleure testabilité du code.

Dans notre projet, deux services principaux sont utilisés ici : un pour la gestion des utilisateurs (lié à Spring Security), et un autre pour le traitement des messages MQTT reçus des capteurs connectés.

Extrait significatif

```
@Override  
public UserDetails loadUserByUsername(String username)  
    throws UsernameNotFoundException {
```

```

    User user = userRepository.findByUsername(username);
    if (user == null) {
        throw new UsernameNotFoundException("Utilisateur non trouvé");
    }
    return new org.springframework.security.core.userdetails.User(
        user.getUsername(),
        user.getPassword(),
        Collections.singletonList(
            new SimpleGrantedAuthority("ROLE_" + user.getRole())
        )
    );
}

```

Ce service permet :

- De valider les identifiants fournis à la connexion.
- D'attribuer dynamiquement un rôle à l'utilisateur (ROLE_USER, ROLE_ADMIN).
- D'interfacer Spring Security avec notre base PostgreSQL.

2. MqttService.java – Service de gestion des messages capteurs

Ce service écoute en continu les données envoyées par les capteurs via le protocole MQTT. Il utilise la bibliothèque Eclipse Paho pour s'abonner au topic défini dans le fichier `application.properties`.

Connexion et abonnement au topic

```

@PostConstruct
public void init() {
    connect(); // Appelé automatiquement au lancement
}

public void connect() {
    client = new MqttClient(brokerUrl, clientId, new MemoryPersistence());
    ...
    client.subscribe(topic); // Ex: esp32/sensors
}

```

Une fois connecté, il traite chaque message MQTT comme suit :

Traitement d'un message JSON reçu

```
public void messageArrived(String topic, MqttMessage message) {  
    JSONObject json = new JSONObject(new String(message.getPayload()));  
    float temperature = (float) json.getDouble("temperature");  
    ...  
    String macAddress = json.optString("macAddress", null);  
    // Enregistrement en base  
}
```

Ce service :

- Parse les données envoyées par les capteurs ESP32 (JSON).
- Associe chaque mesure à un capteur via son `deviceId`.
- Enregistre les mesures dans la base PostgreSQL via les repositories.

Rôle général des services

Les services assurent :

- Une séparation propre entre logique métier et interface REST.
- La réutilisabilité et la testabilité des traitements.
- La centralisation des opérations complexes (vérifications, calculs, gestion de droits...).

Note : Les services liés à la sécurité JWT (génération, validation de token, configuration HTTPS) seront détaillés dans le chapitre dédié à la **sécurisation de l'API REST**.

Remarque : Le code complet des services est fourni en annexe, avec des commentaires expliquant chaque étape importante pour une meilleure compréhension.

Les DTO (Data Transfer Object)

Dans une architecture REST comme celle utilisée dans notre projet, les **DTO** (Data Transfer Object) sont des classes intermédiaires servant à transporter des données entre le backend et le frontend, sans exposer directement les entités de la base de données. Cela permet de :

- ne transférer que les données strictement nécessaires ;
- garantir la sécurité (en évitant d'exposer des champs sensibles comme les mots de passe) ;
- structurer les données selon les besoins du client (frontend PHP).

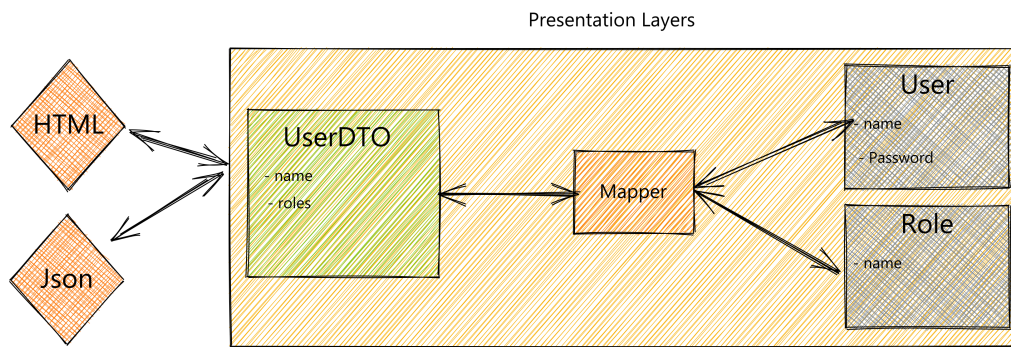


FIGURE 6.6 – illustration d'un DTO

PermissionDTO Ce DTO est utilisé par l'administrateur pour définir ou modifier les permissions d'accès d'un utilisateur (capteurs visibles ou non). Il encapsule quatre attributs booléens :

- canViewTemperature
- canViewHumidity
- canViewLuminosite
- canViewHumiditeSol

Extrait significatif

```
public class PermissionDTO {
    private boolean canViewTemperature;
    private boolean canViewHumidity;
    private boolean canViewLuminosite;
    private boolean canViewHumiditeSol;
    // Getters et Setters
}
```

Utilisation : Ce DTO est utilisé dans la méthode `updateUserPermissions()` du `AdminController` pour ajuster dynamiquement les droits d'accès de chaque utilisateur.

MeasurementDTO Ce DTO est utilisé pour retourner au client uniquement les mesures auxquelles un utilisateur a le droit d'accéder, en fonction de ses permissions. Il contient :

- Les mesures : température, humidité, luminosité brute, humidité du sol brute
- Le moment de la mesure (timestamp)
- L'identifiant du capteur (deviceId)

Extrait significatif

```
public class MeasurementDTO {  
    private Float temperature;  
    private Float humidity;  
    private Integer luminositeRaw;  
    private Integer humiditeSolRaw;  
    private LocalDateTime timestamp;  
    private String deviceId;  
    // Getters et Setters  
}
```

Utilisation : Ce DTO est utilisé dans le `MeasurementController` pour renvoyer au frontend les données filtrées selon les droits de chaque utilisateur connecté, via l'endpoint `/measurements/mine`.

Ces DTO garantissent une communication propre, sécurisée et adaptée entre le backend et l'interface utilisateur.

7 Interface Web (Frontend PHP)

L'interface utilisateur du projet a été développée en **PHP** avec du **HTML**, du **CSS** et un peu de **JavaScript**. Elle permet aux utilisateurs :

- de se connecter à leur compte,
- de visualiser les dernières données environnementales,
- et pour les administrateurs, de gérer les utilisateurs et leurs autorisations.

1. Page de connexion (**login.php**)

Cette page contient un formulaire HTML stylisé, dans lequel l'utilisateur saisit son nom d'utilisateur et son mot de passe. Une fois les données saisies, un script JavaScript les envoie au backend :

- une requête POST est envoyée vers `/api/auth/login` (backend Spring Boot) ;
- en cas de succès, un **jeton d'authentification** et le rôle sont retournés ;
- ces données sont ensuite transmises à `do_login.php` pour enregistrer la session PHP.

2. Création de session (**do_login.php**)

Ce fichier joue un rôle de **pont entre le backend et la session PHP**. Il :

- reçoit les données (`username`, `token`, `role`) ;
- vérifie la validité du jeton via une requête GET vers `/api/auth/me` ;
- si le jeton est valide, il enregistre les données dans la session PHP via `$_SESSION` ;
- sinon, il renvoie une erreur JSON.

Cette vérification permet de s'assurer que seul un utilisateur réellement authentifié puisse établir une session.

3. Tableau de bord (**dashboard.php**)

Une fois connecté, l'utilisateur est redirigé vers le **tableau de bord**. Cette page :

- récupère les dernières mesures via une requête cURL vers l'API ;
- les affiche sous forme de cartes : température, humidité, luminosité, humidité du sol ;
- trace également un **graphique dynamique** à l'aide de `Chart.js` ;
- adapte le contenu affiché selon le rôle de l'utilisateur (utilisateur simple ou administrateur).

4. Interface d'administration (**admin_users.php**)

Accessible uniquement aux utilisateurs de rôle ADMIN, cette page permet :

- la création de nouveaux utilisateurs ;
- l'assignation ou la suppression d'un capteur pour chaque utilisateur ;

- la modification des autorisations d'accès à chaque type de mesure (via des cases à cocher) ;
- la suppression d'un utilisateur.

Toutes les actions sont effectuées via des requêtes asynchrones (AJAX) vers l'API REST sécurisée, en utilisant le **jeton d'accès enregistré dans la session**.

5. Contrôle d'accès

Chaque page sécurisée commence par une vérification PHP comme ci-dessous :

```
session_start();
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit();
}
```

Cela permet d'éviter qu'un utilisateur non connecté accède à des données sensibles. En cas d'absence de session valide, l'utilisateur est automatiquement redirigé vers la page de connexion.

6. Résumé de la structure côté client

Fichier	Rôle
login.php	Affiche le formulaire de connexion et envoie les identifiants.
do_login.php	Vérifie le jeton reçu et crée la session PHP.
dashboard.php	Affiche les dernières mesures et un graphique dynamique.
admin_users.php	Permet à l'administrateur de gérer les utilisateurs et leurs capteurs.
logout.php	Déconnecte l'utilisateur et détruit la session.

Cette interface simple mais fonctionnelle permet à tous les utilisateurs de suivre l'évolution des mesures en temps réel et aux administrateurs de gérer l'accès aux capteurs. La communication avec le backend est assurée via une API REST sécurisée, qui sera détaillée dans la partie suivante dédiée à la sécurité.

8 Sécurité de l'application IoT

Avant d'intégrer notre propre système de sécurité, nous avons réalisé une série d'entraînements sur deux plateformes web volontairement vulnérables : extbfDVWA (Damn Vulnerable Web Application) et extbfJuice Shop (OWASP). Cela nous a permis d'expérimenter divers types d'attaques et de mieux comprendre les mécanismes de défense à mettre en place.

Nous avons réalisé des tests en boîte blanche et boîte noire, en reproduisant des attaques telles que : SQL Injection (classique et blind), XSS, CSRF, brute-force, injection de commande, file inclusion, etc.

1. Spring Security

Spring Security est un framework de sécurisation des applications Spring Boot. Il fournit les outils pour gérer l'authentification, l'autorisation, la protection contre les attaques CSRF, CORS et bien plus. Dans notre projet, Spring Security est configuré pour :

- Désactiver les sessions (mode STATELESS), ce qui impose l'utilisation des jetons JWT pour chaque requête.
- Restreindre l'accès aux routes `/api/admin/users/**` uniquement aux administrateurs.
- Autoriser librement l'accès à certaines routes publiques comme `/api/auth/login` ou `/api/auth/register`.
- Ajouter un filtre personnalisé `JwtAuthFilter` pour valider chaque requête entrante.

2. Authentification avec JWT (JSON Web Token)

Les jetons JWT permettent de s'assurer de l'identité de l'utilisateur sans stocker d'informations en session serveur. Le fonctionnement est le suivant :

- Lors de la connexion, un jeton est généré contenant : l'identifiant de l'utilisateur, une date d'expiration, et une signature.
- Ce jeton est signé avec une clé secrète (stockée en base64 dans `application.properties`).
- Le filtre `JwtAuthFilter` extrait le jeton à chaque requête entrante et vérifie :
 - Que le token est présent et commence par `Bearer` .
 - Qu'il n'est pas expiré.
 - Qu'il correspond bien à l'utilisateur authentifié.

Fonctionnement du filtre `JwtAuthFilter` :

- Il hérite de `OncePerRequestFilter` pour s'exécuter une seule fois par requête.
- Il vérifie la présence d'un jeton JWT dans l'en-tête `Authorization`.
- Si le jeton est valide, il initialise le contexte de sécurité avec l'utilisateur authentifié.

3. Le CORS : Cross-Origin Resource Sharing

CORS est un mécanisme de sécurité des navigateurs qui empêche un site web d'effectuer des requêtes HTTP vers un autre domaine sans autorisation. Pour contourner cette restriction légitime, on doit définir une politique CORS dans le backend (ici Spring Boot) qui autorise explicitement certaines origines (comme `http://localhost`).

Dans le fichier `SecurityConfig.java`, la configuration CORS autorise les requêtes en provenance de :

- `http://192.168.11.70`, `https://192.168.11.70`
- `http://localhost`, `https://localhost`

4. Chiffrement SSL / HTTPS

L'API REST est accessible via le port 8443 en HTTPS. Le backend Spring Boot utilise un certificat auto-signé pour chiffrer les échanges. Les requêtes côté PHP utilisent l'URL `https://192.168.11.70:8443`.

Remarque : la vérification du certificat SSL est désactivée dans le code PHP via :

```
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
```

5. Configuration de PostgreSQL

Pour s'assurer que seuls certains hôtes puissent accéder à la base de données PostgreSQL, le fichier `pg_hba.conf` a été configuré. On y précise l'adresse locale `192.168.11.0/24` pour autoriser le réseau interne, et `100.82.183.0/24` pour l'accès VPN via Tailscale.

```
# Allow replication connections from localhost, by a user with the
# replication privilege.
local    replication    all                                     peer
host     replication    all             127.0.0.1/32          scram-sha-256
host     replication    all             ::1/128             scram-sha-256
host     mqtt_data      esp32_user      192.168.11.0/24      md5
host     mqtt_data      esp32_user      100.82.183.0/24      md5
```

FIGURE 8.1 – Extrait du fichier `pg_hba.conf` autorisant l'accès à l'utilisateur `esp32_user` depuis des adresses IP précises (LAN et VPN)

6. DTO et validation des données

Les DTO (Data Transfer Objects) sont utilisés pour transporter les données de manière sécurisée :

- Ils permettent de filtrer les champs entrants et sortants.
- On évite ainsi d'exposer des informations sensibles (ID, hash...).
- Cela ajoute une couche de validation supplémentaire.

7. Pare-feu et accès réseau

Un pare-feu est présent sur la Raspberry Pi pour contrôler les ports ouverts. Idéalement :

- Seuls les ports 8443 (HTTPS) et 1883 (MQTT) sont ouverts.
- Tous les autres ports sont bloqués par défaut.
- Un outil comme ufw (Uncomplicated Firewall) est utilisé pour gérer les ports.

8. Cross-Site Request Forgery (CSRF)

CSRF est une attaque qui exploite la confiance d'un utilisateur connecté pour envoyer des requêtes malicieuses à sa place. Par exemple, si un utilisateur authentifié clique sur un lien malicieux, son navigateur peut envoyer une requête à l'application ciblée avec ses cookies d'authentification.

Dans notre cas, comme l'authentification est gérée via JWT dans les en-têtes et non via des cookies, nous sommes naturellement protégés contre les attaques CSRF. De plus, Spring Security désactive CSRF pour les API REST dans notre configuration.

9. Autres protections en place

- Les mots de passe sont hachés avec BCrypt (fort contre rainbow tables).
- Le frontend PHP utilise des sessions pour limiter l'accès à l'interface.
- Les erreurs API sont minimisées pour ne pas exposer d'informations sensibles.
- Les injections SQL sont prévenues par l'utilisation de JPA/Hibernate avec des requêtes paramétrées et des DTO qui valident les entrées.

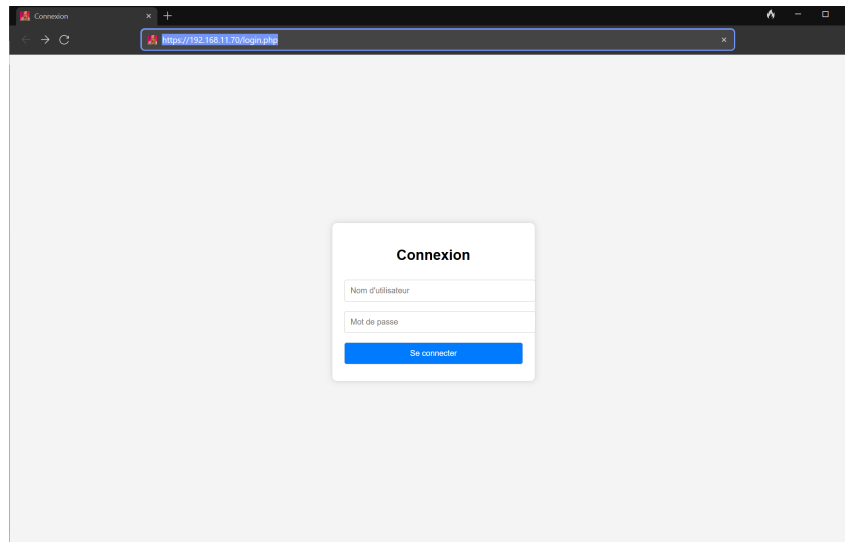


FIGURE 9.2 – Interface de connexion utilisateur

```

text path '/'
2025-03-20T16:15:28.889+01:00 INFO 1140 --- [main] c.e.api.esp32_mqtt.Esp32MqttApplication : Started Esp32MqttApplication in 7.842 second
s (process running for 8.199)
Message reçu : {"temperature":20.0,"humidity":68.0,"luminosite_raw":0,"humidite_sol_raw":4095,"macAddress":"94:B9:7E:EB:18:14"}
Hibernate:
select
  d1_0.id,
  d1_0.device_id,
  d1_0.name,
  d1_0.user_id
from
  devices d1_0
where
  d1_0.device_id=?
Hibernate:
insert
into
  measurements
(device_id, humidite_sol_raw, humidity, luminosite_raw, temperature, timestamp)
values
(?, ?, ?, ?, ?, ?)

```

FIGURE 9.3 – Traitement d’une trame MQTT et insertion automatique en base de données

- Créer de nouveaux comptes utilisateurs ;
- Associer un capteur (deviceId) à chaque utilisateur ;
- Gérer les permissions d’accès aux types de données (température, humidité...) ;
- Supprimer ou modifier les comptes existants.

Toutes les actions sont réalisées via des requêtes AJAX sécurisées, grâce au token JWT stocké en session PHP.

6. Visualisation des changements en base PostgreSQL

Pour confirmer que les données envoyées par les capteurs sont bien stockées, une vérification a été effectuée directement dans la base de données PostgreSQL via l’outil en ligne de commande ‘psql’.

La base contient notamment les tables suivantes :

- **users** : stocke les identifiants et rôles des utilisateurs ;
- **devices** : contient les capteurs identifiés par leur adresse MAC ;
- **measurements** : enregistre les données mesurées (température, humidité, etc.) avec horodatage ;
- **user_permissions** : définit les droits d’accès par utilisateur.

Ces tests confirment que toute la chaîne est fonctionnelle, du capteur à l’affichage web, avec une gestion fine des permissions et une communication sécurisée via HTTPS et JWT.

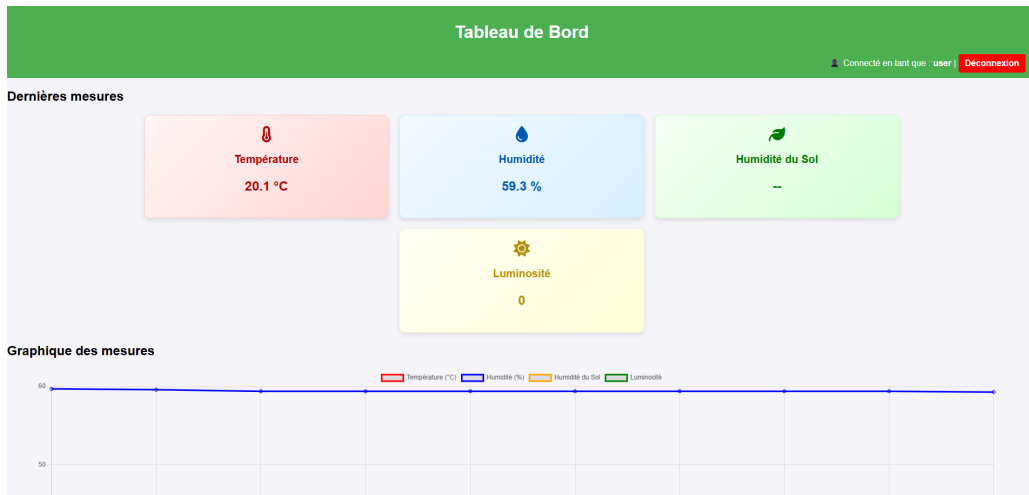


FIGURE 9.4 – Affichage des mesures sur le tableau de bord utilisateur

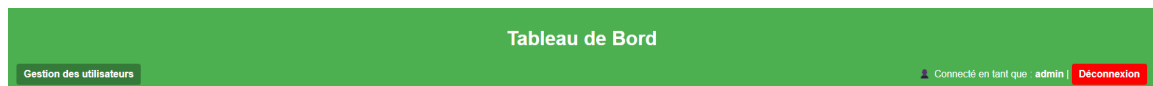


FIGURE 9.5 – Gestion des utilisateurs, assignation de capteurs et droits d'accès

Menu principal | Gestion des utilisateurs | Déconnexion

Gestion des utilisateurs

Créer un nouvel utilisateur

Nom d'utilisateur: [] Mot de passe: [] Créer

Liste des utilisateurs

Nom	Rôle	Capteur Assigné	Température	Humidité	Luminosité	Humidité Sol	Actions
admin	ADMIN	-- Aucun --	☐	☐	☐	☐	Enregistrer Supprimer
voiture	USER	-- Aucun --	☒	☐	☒	☒	Enregistrer Supprimer
test	USER	-- Aucun --	☒	☒	☒	☐	Enregistrer Supprimer
user	USER	94B97E EB 18 14	☒	☒	☒	☐	Enregistrer Supprimer

FIGURE 9.6 – Barre de navigation de l'interface administrateur

Schema	Name	Type	Owner
public	devices	table	esp32_user
public	devices_id_seq	sequence	esp32_user
public	measurements	table	esp32_user
public	measurements_id_seq	sequence	esp32_user
public	user_permissions	table	esp32_user
public	user_permissions_id_seq	sequence	esp32_user
public	users	table	esp32_user
public	users_id_seq	sequence	esp32_user

(8 rows)

FIGURE 9.7 – Liste des tables présentes dans la base mqtt_data

```

admin@server:~$ sudo -u postgres psql
psql (15.10 (Debian 15.10-0+deb12u1))
Type "help" for help.

postgres=# \c mqtt_data
You are now connected to database "mqtt_data" as user "postgres".
mqtt_data=# select * from users;
 id | password | role | username
-----+-----+-----+-----
  1 | $2a$10$I59mFdRjWIaxwEneZeq0GOSyutZgIR06lWH0Q5hTRT.VCybME5SF2 | ADMIN | admin
  3 | $2a$10$a1HVsz10RdL6IF8rWvQd2.tjSXXbJi6bUr8nBq5xeeGIeIggGXCKs | USER | voiture
  4 | $2a$10$Wb9mIFpP9WUzkHquQ82Na0tgjwSr1fVL8ny01waT9TZ7cJPByzgu2 | USER | test
  5 | $2a$10$MG0GesDfIA/j4BE3JXo9Wu2qfGDjhUC5Eibj9t2/Htd9xbNk/kDG | USER | user
(4 rows)

mqtt_data=#

```

FIGURE 9.8 – Contenu de la table users avec les identifiants chiffrés et les rôles

id	humidite_sol_raw	humidity	luminosite_raw	temperature	timestamp	device_id
1	4095	37.4	165	18.4	2025-03-14 16:15:14.053067	1
2	4095	37.4	167	18.4	2025-03-14 16:15:15.829946	1
3	4095	37.3	160	18.4	2025-03-14 16:15:17.8392	1
4	4095	37.2	163	18.4	2025-03-14 16:15:19.854544	1
5	4095	37.2	155	18.4	2025-03-14 16:15:21.870912	1
6	4095	37.1	166	18.4	2025-03-14 16:15:23.887382	1
7	4095	37.1	162	18.4	2025-03-14 16:15:25.902924	1
8	4095	37.1	160	18.4	2025-03-14 16:15:27.918262	1
9	4095	37.1	167	18.4	2025-03-14 16:15:29.934358	1
10	4095	37.1	170	18.4	2025-03-14 16:15:31.950873	1
11	4095	37.2	165	18.4	2025-03-14 16:15:33.965861	1
12	4095	37.2	163	18.4	2025-03-14 16:15:35.983054	1
13	4095	37.2	161	18.4	2025-03-14 16:15:37.997692	1
14	4095	37.2	162	18.4	2025-03-14 16:15:40.014595	1
15	4095	37.2	166	18.4	2025-03-14 16:15:42.026864	1
16	4095	37.3	156	18.4	2025-03-14 16:15:44.045846	1
17	4095	37.4	160	18.4	2025-03-14 16:15:46.061149	1
18	4095	37.4	163	18.4	2025-03-14 16:15:48.079753	1
19	4095	37.4	163	18.4	2025-03-14 16:15:50.096896	1
20	4095	37.4	165	18.4	2025-03-14 16:15:52.11	1
21	4095	37.5	166	18.4	2025-03-14 16:15:54.125895	1
22	4095	37.4	163	18.4	2025-03-14 16:15:56.142889	1
23	4095	37.3	162	18.4	2025-03-14 16:15:58.157864	1
24	4095	37.3	173	18.4	2025-03-14 16:16:00.174288	1
25	4095	37.3	162	18.4	2025-03-14 16:16:02.190291	1
26	4095	37.3	158	18.4	2025-03-14 16:16:04.206455	1
27	4095	37.4	161	18.5	2025-03-14 16:16:06.222599	1

FIGURE 9.9 – Exemples de mesures enregistrées automatiquement dans la table measurements

```

mqtt_data=# \d user_permissions
Table "public.user_permissions"
  Column          | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id               | bigint       |           | not null | generated by default as identity
 can_view_humidite_sol | boolean      |           |          |
 can_view_humidity   | boolean      |           |          |
 can_view_luminosite | boolean      |           |          |
 can_view_temperature | boolean      |           |          |
 user_id          | bigint       |           |          |
Indexes:
    "user_permissions_pkey" PRIMARY KEY, btree (id)
    "uk525n7wn9ejoa648m26bm4rhtt" UNIQUE CONSTRAINT, btree (user_id)
Foreign-key constraints:
    "fkrowth8b2bngrxd1gafh13005u" FOREIGN KEY (user_id) REFERENCES users(id)

mqtt_data=#

```

FIGURE 9.10 – Structure de la table user_permissions, avec les colonnes de droits d'accès

Column		Type	Table "public.devices"		Default
			Collation	Nullable	
id		bigint		not null	generated by default as identity
device_id		character varying(255)		not null	
name		character varying(255)			
user_id		bigint			

Indexes:

- "devices_pkey" PRIMARY KEY, btree (id)
- "ukkl1lviut5wsyxgnd1t3wqf9hbu" UNIQUE CONSTRAINT, btree (device_id)

Foreign-key constraints:

- "fkrfbrilymrwywdydc4dgywelbt" FOREIGN KEY (user_id) REFERENCES users(id)

Referenced by:

- TABLE "measurements" CONSTRAINT "fk5lg8jeeoo5ii3ljqm2n9us1bo" FOREIGN KEY (device_id) REFERENCES devices(id)

FIGURE 9.11 – Structure de la table `devices`, avec les relations vers les utilisateurs

10 Difficultés rencontrées

Durant l'intégration complète du système, plusieurs difficultés techniques ont été rencontrées :

- **SPI et Concentrateur LoRa** : Déjà évoqué précédemment, la configuration du bus SPI et le câblage des broches pour activer le concentrateur LoRa sur la Raspberry Pi ont nécessité un travail d'identification et d'ajustement précis, notamment sur les broches CE/CS, RESET et DIO.
- **Configuration réseau et exposition du site web** : L'une des difficultés majeures a été la volonté d'exposer notre site web (hébergé sur la Raspberry Pi 5) sur Internet afin de le rendre accessible depuis l'extérieur. Plusieurs tentatives ont été menées :
 - **Tentative avec DuckDNS + port forwarding (redirection de port)** : DuckDNS est un service de DNS dynamique permettant d'associer une adresse IP publique dynamique à un nom de domaine. L'idée était de configurer un nom de domaine comme `monprojet.duckdns.org` et de rediriger les ports HTTP/HTTPS vers la Raspberry Pi à travers la box. Cependant, cette méthode nécessite :
 - une adresse IP publique non partagée (ce qui n'est pas le cas avec un forfait 4G) ;
 - la possibilité d'ouvrir des ports sur la box (ce qui est interdit sur les réseaux de type NAT CGNAT ou en environnement académique).Malgré plusieurs tests, cela s'est soldé par un échec : le réseau 4G utilise le CGNAT (Carrier Grade NAT), empêchant toute redirection de port, et le réseau du campus interdit les connexions entrantes.
 - **Essai sur réseau local du campus** : Même en connectant la Raspberry Pi à l'Ethernet du campus, les connexions entrantes sont filtrées par le pare-feu de l'infrastructure réseau. Aucun port ne pouvait être ouvert ni redirigé.
 - **Solution fonctionnelle avec NGROK** : Finalement, la solution retenue a été l'utilisation de **NGROK**. Il s'agit d'un outil de tunneling réseau permettant de créer un tunnel sécurisé vers une machine locale à travers Internet. NGROK crée une URL publique (ex : `https://xyz.ngrok.io`) qui redirige automatiquement vers le port local de la Raspberry Pi. Il contourne ainsi les limitations du NAT et du pare-feu :
 - Pas besoin d'adresse IP publique ;
 - Aucun port à ouvrir sur le routeur ;
 - Compatible même sur réseau mobile ou académique restreint.

Grâce à NGROK, notre API REST (port 8080) et notre site PHP (port 80) ont pu être exposés à distance sans configuration réseau complexe.

- **Authentication JWT** : L'intégration d'un système d'authentification par token JWT a également nécessité une phase de débogage. Il a fallu s'assurer que :
 - le token était bien généré et retourné au frontend lors du login ;
 - le token était transmis dans les headers des requêtes suivantes ;
 - le filtre Spring Security intercepte correctement les requêtes ;
 - l'expiration du token est gérée proprement côté PHP (session et redirection).

Solutions mises en œuvre

En résumé, plusieurs solutions ont été mises en place :

- Utilisation de **NGROK** pour exposer le backend et le frontend sans ouvrir de ports ;
- Gestion complète des autorisations via une API REST sécurisée par JWT ;
- Interface d'administration fonctionnelle pour gérer utilisateurs, permissions et capteurs ;
- Hébergement local fiable sur la Raspberry Pi 5 avec accès distant sécurisé pour les tests.

11 Perspectives et Améliorations

Le système IoT mis en place offre une solution complète allant de la mesure environnementale à l’affichage sécurisé sur une interface web. Toutefois, plusieurs pistes concrètes d’amélioration ont été identifiées, tant au niveau fonctionnel que technique :

- **Finaliser l’intégration LoRaWAN avec The Things Network (TTN)**
À l’heure actuelle, les communications LoRa sont réalisées en mode point à point (P2P), sans infrastructure réseau. L’objectif est de connecter la passerelle LoRa (Raspberry Pi 3B + module RHF0M301) au réseau **LoRaWAN** via **The Things Network (TTN)**.

TTN est un réseau LoRaWAN mondial et collaboratif, qui fournit l’infrastructure nécessaire à l’authentification, la gestion des nœuds et la sécurité des données (via chiffrement AES128). En le connectant :

- Les capteurs LoRa pourront être enregistrés comme *end devices* dans TTN ;
- La passerelle agira comme un relai vers le réseau LoRaWAN mondial, en utilisant un lien UDP (Packet Forwarder) ;
- TTN transférera les données vers notre serveur applicatif (Spring Boot) via un **webhook HTTP/MQTT**.

Cela permettrait une meilleure scalabilité, un suivi des trames et un envoi optimisé dans un contexte réel.

- **Connexion de la passerelle au réseau local de l’établissement**

L’un des objectifs était de connecter la passerelle LoRa au réseau local du campus pour accéder à la base de données distante (hébergée sur la Raspberry Pi 5). Toutefois, des restrictions réseau (notamment le blocage des connexions entrantes) ont empêché cette communication.

L’idée serait :

- Soit de demander l’ouverture temporaire de ports spécifiques pour autoriser le trafic MQTT ou HTTP vers l’API ;
 - Soit d’utiliser un tunnel VPN chiffré (ex. WireGuard, Tailscale) pour relier la passerelle au serveur, en contournant les restrictions NAT ;
 - Soit d’exploiter une instance TTN comme passerelle intermédiaire entre les capteurs et le backend.
- **Exploiter le projet de sauvegarde en réseau mis en place en classe**
Dans le cadre des TPs précédents, un projet de **NAS (Network Attached Storage)** avait été configuré avec des scripts de sauvegarde automatisés. Ce système pourrait être directement relié à notre Raspberry Pi serveur pour :
 - Réaliser des **backups réguliers** de la base PostgreSQL (ex. via `pg_dump`) ;
 - Sauvegarder les fichiers de configuration et certificats HTTPS/JWT ;
 - Archiver les logs applicatifs pour analyse ultérieure.

- **Effectuer des tests de sécurité avancés (Pentest)**

Dans une phase ultérieure, une analyse approfondie de la sécurité du système pourrait être menée via :

- Des tests d'injection (SQLi, XSS) sur les formulaires de connexion et d'administration PHP ;
- Des tests de robustesse du serveur MQTT (ex. attaques de flood ou injection de trames) ;
- La mise en place d'un script d'attaque automatisée (ex. avec OWASP ZAP ou Burp Suite) pour simuler un attaquant externe ;
- L'ajout de logs d'authentification JWT pour détecter les tentatives non autorisées.

L'objectif étant de sécuriser davantage l'environnement de production avant un éventuel déploiement public.

Ces perspectives permettent de faire évoluer le projet vers une solution IoT complète, sécurisée, maintenable et potentiellement déployable à grande échelle, dans un cadre académique ou professionnel.

12 Conclusion

Ce projet nous a permis de concevoir et de mettre en œuvre un système IoT complet, depuis la collecte des données capteur avec des ESP32, jusqu'à leur visualisation sur une interface web sécurisée. Nous avons intégré une architecture technique incluant un backend Java Spring Boot protégé par JWT, un broker MQTT, une base de données PostgreSQL, et une interface web en PHP.

Tout au long du développement, nous avons porté une attention particulière à la sécurité, à la modularité et à la gestion des permissions. Malgré des difficultés techniques — notamment autour de la configuration LoRa, des limitations réseau liées à la 4G et au campus, et du déploiement sur internet — nous avons réussi à obtenir un prototype fonctionnel, évolutif et prêt à être amélioré.

En revanche, nous avons sous-estimé l'ampleur du projet, en particulier le temps nécessaire pour développer une API robuste et sécurisée. Cela nous a empêchés de consacrer autant de temps que prévu à l'aspect pentest, qui faisait pourtant partie de nos objectifs initiaux.

Ce projet nous a offert une expérience concrète, riche en apprentissages, mêlant programmation, sécurité, électronique et communication réseau. Il ouvre la voie à de nombreuses améliorations futures.

13 Annexes

Dans les annexes, nous présentons les fonctions importantes de notre application.

Back-end

Annexe a) Controllers

```
1 // === CONTROLLER: AdminUserController ===
2
3 /**
4  * Crée un utilisateur avec un rôle et des permissions par défaut.
5  */
6 @PostMapping("/create")
7 public ResponseEntity<?> createUser(@RequestBody User user) {
8     if (userRepository.findByUsername(user.getUsername()) != null) {
9         return ResponseEntity.status(HttpStatus.CONFLICT).body("Utilisateur déjà existant");
10    }
11    user.setPassword(passwordEncoder.encode(user.getPassword()));
12    user.setRole(user.getRole() == null ? "USER" : user.getRole());
13
14    UserPermission permission = new UserPermission();
15    permission.setUser(user);
16    permission.setCanViewTemperature(true);
17    permission.setCanViewLuminosite(true);
18    user.setPermission(permission);
19
20    userRepository.save(user);
21    return ResponseEntity.status(HttpStatus.CREATED).body("Utilisateur créé avec succès");
22 }
23
24 /**
25  * Met à jour les permissions d'un utilisateur existant.
26  */
27 @PutMapping("/update-permissions/{username}")
28 public ResponseEntity<?> updateUserPermissions(@PathVariable String username, @RequestBody
    PermissionDTO dto) {
29     User user = userRepository.findByUsername(username);
30     if (user == null) return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Utilisateur
        introuvable");
31
32     UserPermission p = user.getPermission();
33     if (p == null) {
34         p = new UserPermission();
35         p.setUser(user);
36         user.setPermission(p);
37     }
38     p.setCanViewTemperature(dto.isCanViewTemperature());
39     p.setCanViewHumidity(dto.isCanViewHumidity());
40     p.setCanViewLuminosite(dto.isCanViewLuminosite());
41     p.setCanViewHumiditeSol(dto.isCanViewHumiditeSol());
42
43     userRepository.save(user);
44     return ResponseEntity.ok("Permissions mises à jour");
45 }
46
47 // === CONTROLLER: AuthController ===
48
49 /**
50  * Authentifie un utilisateur et retourne un token JWT.
51  */
52 @PostMapping("/login")
53 public ResponseEntity<?> login(@RequestBody User loginRequest) {
54     User user = userRepository.findByUsername(loginRequest.getUsername());
55     if (user == null || !passwordEncoder.matches(loginRequest.getPassword(),
        user.getPassword())) {
56         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Identifiants incorrects");
57     }
58     UserDetails userDetails = userDetailsService.loadUserByUsername(user.getUsername());
59     String token = jwtService.generateToken(userDetails);
```

```

60     return ResponseEntity.ok(Map.of("token", token, "username", user.getUsername(), "role",
61                                     user.getRole()));
62 }
63 // === CONTROLLER: DeviceController ===
64
65 /**
66  * Associe un capteur à un utilisateur.
67  */
68 @PostMapping("/assign")
69 public ResponseEntity<String> assignDeviceToUser(@RequestParam String deviceId, @RequestParam
70         String username) {
71     Device device = deviceRepository.findById(deviceId);
72     User user = userRepository.findByUsername(username);
73     if (device == null || user == null) return ResponseEntity.badRequest().body("Erreur
74         d'association");
75
76     device.setUser(user);
77     deviceRepository.save(device);
78     return ResponseEntity.ok("Device associé à l'utilisateur");
79 }
80
81 /**
82  * Récupère les mesures de l'utilisateur connecté.
83  */
84 @GetMapping("/mine-measurements")
85 public ResponseEntity<List<Measurement>> getUserMeasurements(Authentication auth) {
86     User user = userRepository.findByUsername(auth.getName());
87     List<Measurement> result = new ArrayList<>();
88     for (Device d : deviceRepository.findByUser(user)) {
89         result.addAll(measurementRepository.findByDevice(d));
90     }
91     return result.isEmpty() ? ResponseEntity.noContent().build() : ResponseEntity.ok(result);
92 }
93
94 // === CONTROLLER: MeasurementController ===
95
96 /**
97  * Récupère les mesures visibles d'un utilisateur connecté selon ses permissions.
98  */
99 @GetMapping("/mine")
100 public ResponseEntity<List<MeasurementDTO>> getUserMeasurementsDTO(Authentication auth) {
101     User user = userRepository.findByUsername(auth.getName());
102     List<MeasurementDTO> dtos = deviceRepository.findByUser(user).stream()
103         .flatMap(d -> measurementRepository.findByDevice(d).stream())
104         .map(m -> {
105             MeasurementDTO dto = new MeasurementDTO();
106             dto.setTimestamp(m.getTimestamp());
107             dto.setDeviceId(m.getDevice().getDeviceId());
108             if (user.getPermission().isCanViewTemperature())
109                 dto.setTemperature(m.getTemperature());
110             if (user.getPermission().isCanViewHumidity()) dto.setHumidity(m.getHumidity());
111             if (user.getPermission().isCanViewLuminosite())
112                 dto.setLuminositeRaw(m.getLuminositeRaw());
113             if (user.getPermission().isCanViewHumiditeSol())
114                 dto.setHumiditeSolRaw(m.getHumiditeSolRaw());
115             return dto;
116         }).collect(Collectors.toList());
117     return dtos.isEmpty() ? ResponseEntity.noContent().build() : ResponseEntity.ok(dtos);
118 }

```

Listing 13.1 – Code des controllers

Annexe b) Modèle

```

1 // Résumé des classes du modèle
2
3 // -----
4 // Modèle : User
5 // -----
6 @Entity
7 @Table(name = "users")
8 public class User {

```

```

9      @Id
10      @GeneratedValue(strategy = GenerationType.IDENTITY)
11      private Long id;
12
13      @Column(unique = true, nullable = false)
14      private String username; // Nom d'utilisateur unique
15
16      @Column(nullable = false)
17      private String password; // Mot de passe de l'utilisateur (crypté)
18
19      @Column(nullable = false)
20      private String role = "USER"; // Rôle par défaut : USER
21
22      @OneToOne(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
23      private UserPermission permission; // Permissions spécifiques à l'utilisateur
24
25      // Getters et Setters...
26  }
27
28  // -----
29  // Modèle : UserPermission
30  // -----
31
32  @Entity
33  @Table(name = "user_permissions")
34  public class UserPermission {
35      @Id
36      @GeneratedValue(strategy = GenerationType.IDENTITY)
37      private Long id;
38
39      @OneToOne
40      @JoinColumn(name = "user_id", unique = true)
41      private User user; // Lien avec l'utilisateur
42
43      private boolean canViewTemperature; // Peut voir la température ?
44      private boolean canViewHumidity; // Peut voir l'humidité ?
45      private boolean canViewLuminosity; // Peut voir la luminosité ?
46      private boolean canViewHumiditySol; // Peut voir l'humidité du sol ?
47
48      // Getters et Setters...
49  }
50
51  // -----
52  // Modèle : Device
53  // -----
54
55  @Entity
56  @Table(name = "devices")
57  public class Device {
58      @Id
59      @GeneratedValue(strategy = GenerationType.IDENTITY)
60      private Long id;
61
62      @Column(unique = true, nullable = false)
63      private String deviceId; // Identifiant unique (ex: adresse MAC)
64
65      private String name; // Nom ou label du capteur
66
67      @ManyToOne(fetch = FetchType.LAZY, optional = true)
68      @JoinColumn(name = "user_id")
69      private User user; // Utilisateur associé
70
71      @OneToMany(mappedBy = "device", cascade = CascadeType.ALL, orphanRemoval = true)
72      private List<Measurement> measurements; // Mesures associées à ce capteur
73
74      // Getters et Setters...
75  }
76
77  // -----
78  // Modèle : Measurement
79  // -----
80
81  @Entity
82  @Table(name = "measurements")
83  public class Measurement {
84      @Id

```

```

85     @GeneratedValue(strategy = GenerationType.IDENTITY)
86     private Long id;
87
88     private float temperature;    // Température mesurée
89     private float humidity;       // Humidité mesurée
90     private int luminositeRaw;    // Valeur brute de luminosité
91     private int humiditeSolRaw;   // Valeur brute d'humidité du sol
92
93     private LocalDateTime timestamp = LocalDateTime.now(); // Date/heure de la mesure
94
95     @ManyToOne(fetch = FetchType.LAZY)
96     @JoinColumn(name = "device_id")
97     private Device device; // Capteur associé
98
99     // Getters et Setters...
100 }

```

Listing 13.2 – Résumé des classes du modèle

Annexe c) Services

```

1  // == 1. CustomUserDetailsService.java ==
2  // Service utilisé par Spring Security pour charger un utilisateur depuis la base de données
3  @Service
4  public class CustomUserDetailsService implements UserDetailsService {
5
6      @Autowired
7      private UserRepository userRepository;
8
9      @Override
10     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
11         User user = userRepository.findByUsername(username);
12         if (user == null) {
13             throw new UsernameNotFoundException("Utilisateur non trouvé");
14         }
15         return new org.springframework.security.core.userdetails.User(
16             user.getUsername(),
17             user.getPassword(),
18             Collections.singletonList(new SimpleGrantedAuthority("ROLE_" +
19                                     user.getRole()))
20         );
21     }
22 }
23
24 // == 2. JwtService.java ==
25 // Service qui gère la création, validation et lecture des tokens JWT
26 @Service
27 public class JwtService {
28
29     @Value("${jwt.secret}")
30     private String secretString;
31
32     private SecretKey secretKey;
33
34     @PostConstruct
35     public void init() {
36         byte[] keyBytes = Base64.getDecoder().decode(secretString);
37         this.secretKey = Keys.hmacShaKeyFor(keyBytes);
38     }
39
40     public String extractUsername(String token) {
41         return extractClaim(token, Claims::getSubject);
42     }
43
44     public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
45         final Claims claims = extractAllClaims(token);
46         return claimsResolver.apply(claims);
47     }
48
49     private Claims extractAllClaims(String token) {
50         return
51             Jwts.parserBuilder().setSigningKey(secretKey).build().parseClaimsJws(token).getBody();

```

```

51     }
52
53     public String generateToken(UserDetails userDetails) {
54         return generateToken(new HashMap<>(), userDetails);
55     }
56
57     public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {
58         return Jwts.builder()
59             .setClaims(extraClaims)
60             .setSubject(userDetails.getUsername())
61             .setIssuedAt(new Date(System.currentTimeMillis()))
62             .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10))
63             .signWith(secretKey)
64             .compact();
65     }
66
67     public boolean validateToken(String token, UserDetails userDetails) {
68         final String username = extractUsername(token);
69         return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
70     }
71
72     private boolean isTokenExpired(String token) {
73         return extractExpiration(token).before(new Date());
74     }
75
76     private Date extractExpiration(String token) {
77         return extractClaim(token, Claims::getExpiration);
78     }
79 }
80
81
82 // === 3. MqttService.java ===
83 // Service qui écoute le broker MQTT et insère les mesures en base dès réception
84 @Service
85 public class MqttService {
86
87     private final DeviceRepository deviceRepository;
88     private final MeasurementRepository measurementRepository;
89
90     @Value("${mqtt.broker}")
91     private String brokerUrl;
92     @Value("${mqtt.topic}")
93     private String topic;
94     @Value("${mqtt.clientId}")
95     private String clientId;
96     @Value("${mqtt.username}")
97     private String username;
98     @Value("${mqtt.password}")
99     private String password;
100
101     private MqttClient client;
102
103     public MqttService(DeviceRepository deviceRepository, MeasurementRepository
        measurementRepository) {
104         this.deviceRepository = deviceRepository;
105         this.measurementRepository = measurementRepository;
106     }
107
108     @PostConstruct
109     public void init() {
110         connect();
111     }
112
113     public void connect() {
114         try {
115             if (client == null) {
116                 client = new MqttClient(brokerUrl, clientId, new MemoryPersistence());
117             }
118             MqttConnectOptions options = new MqttConnectOptions();
119             options.setUsername(username);
120             options.setPassword(password.toCharArray());
121             options.setAutomaticReconnect(true);
122             options.setCleanSession(true);
123
124             client.setCallback(new MqttCallback() {
125                 @Override

```

```

126     public void connectionLost(Throwable cause) {
127         System.out.println("Connexion MQTT perdue : " + cause.getMessage());
128     }
129
130     @Override
131     public void messageArrived(String topic, MqttMessage message) throws
132         Exception {
133         String payload = new String(message.getPayload());
134         onMessageReceived(payload);
135     }
136
137     @Override
138     public void deliveryComplete(IMqttDeliveryToken token) {
139         System.out.println("Message MQTT envoyé avec succès.");
140     }
141
142     if (!client.isConnected()) {
143         client.connect(options);
144     }
145     client.subscribe(topic);
146 } catch (MqttException e) {
147     e.printStackTrace();
148 }
149 }
150
151 private void onMessageReceived(String payload) {
152     try {
153         JSONObject json = new JSONObject(payload);
154         float temperature = (float) json.getDouble("temperature");
155         float humidity = (float) json.getDouble("humidity");
156         int luminositeRaw = json.getInt("luminosite_raw");
157         int humiditeSolRaw = json.getInt("humidite_sol_raw");
158         String macAddress = json.optString("macAddress", null);
159
160         if (macAddress == null || macAddress.isEmpty()) {
161             throw new IllegalArgumentException("Adresse MAC manquante ou invalide !");
162         }
163
164         Device device = deviceRepository.findById(macAddress);
165         if (device == null) {
166             device = new Device();
167             device.setDeviceId(macAddress);
168             deviceRepository.save(device);
169         }
170
171         Measurement measurement = new Measurement();
172         measurement.setTemperature(temperature);
173         measurement.setHumidity(humidity);
174         measurement.setLuminositeRaw(luminositeRaw);
175         measurement.setHumiditeSolRaw(humiditeSolRaw);
176         measurement.setDevice(device);
177         measurementRepository.save(measurement);
178     } catch (Exception e) {
179         e.printStackTrace();
180     }
181 }
182 }

```

Listing 13.3 – Résumé des services essentiels du backend ESP32 Web

Annexe d) Fonctions essentielles de sécurité

```

1 // =====
2 // AuthController.java (extrait)
3 // =====
4
5 // Authentifie un utilisateur et génère un token JWT
6 @PostMapping("/login")
7 public ResponseEntity<?> login(@RequestBody User loginRequest) {
8     User user = userRepository.findByUsername(loginRequest.getUsername());
9     if (user == null || !passwordEncoder.matches(loginRequest.getPassword(),
10         user.getPassword())) {

```

```

10         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Identifiants invalides");
11     }
12     String token =
13         jwtService.generateToken(userDetailsService.loadUserByUsername(user.getUsername()));
14     return ResponseEntity.ok(Map.of("token", token, "username", user.getUsername(), "role",
15         user.getRole()));
16 }
17 // Retourne les infos de l'utilisateur connecté (authentifié via JWT)
18 @GetMapping("/me")
19 public ResponseEntity<?> getCurrentUser(Authentication auth) {
20     if (auth == null || !auth.isAuthenticated()) {
21         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Non authentifié");
22     }
23     User user = userRepository.findByUsername(auth.getName());
24     return ResponseEntity.ok(Map.of("username", user.getUsername(), "role", user.getRole()));
25 }
26
27 // =====
28 // JwtService.java (extrait)
29 // =====
30
31 // Génère un JWT signé pour un utilisateur donné
32 public String generateToken(UserDetails userDetails) {
33     return Jwts.builder()
34         .setSubject(userDetails.getUsername())
35         .setIssuedAt(new Date())
36         .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10)) // 10h
37         .signWith(secretKey)
38         .compact();
39 }
40
41 // Valide un token JWT (vérifie signature et expiration)
42 public boolean validateToken(String token, UserDetails userDetails) {
43     return extractUsername(token).equals(userDetails.getUsername()) && !isTokenExpired(token);
44 }
45
46 // =====
47 // JwtAuthFilter.java (extrait)
48 // =====
49
50 // Filtre qui intercepte chaque requête pour vérifier le token JWT
51 @Override
52 protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
53     FilterChain chain)
54     throws ServletException, IOException {
55     String authHeader = request.getHeader("Authorization");
56     if (authHeader != null && authHeader.startsWith("Bearer ")) {
57         String token = authHeader.substring(7);
58         String username = jwtService.extractUsername(token);
59         if (username != null && SecurityContextHolder.getContext().getAuthentication() ==
60             null) {
61             UserDetails userDetails = userDetailsService.loadUserByUsername(username);
62             if (jwtService.validateToken(token, userDetails)) {
63                 UsernamePasswordAuthenticationToken authToken = new
64                     UsernamePasswordAuthenticationToken(
65                         userDetails, null, userDetails.getAuthorities());
66                 authToken.setDetails(new
67                     WebAuthenticationDetailsSource().buildDetails(request));
68                 SecurityContextHolder.getContext().setAuthentication(authToken);
69             }
70         }
71     }
72     chain.doFilter(request, response);
73 }
74
75 // =====
76 // SecurityConfig.java (extrait)
77 // =====
78
79 // Chaîne de sécurité définissant les routes publiques, protégées et le filtre JWT
80 @Bean
81 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

```

```

80     http.csrf().disable()
81     .authorizeHttpRequests(auth -> auth
82         .requestMatchers("/api/auth/login", "/api/auth/register").permitAll()
83         .requestMatchers("/api/admin/users/**").hasRole("ADMIN")
84         .anyRequest().authenticated()
85     )
86     .sessionManagement(sess ->
87         sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
87     .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
88     return http.build();
89 }
90
91 // Cryptage des mots de passe avec BCrypt
92 @Bean
93 public PasswordEncoder passwordEncoder() {
94     return new BCryptPasswordEncoder();
95 }

```

Listing 13.4 – Fonctions essentielles de sécurité

Front-end

Vue de redirection : `index.php`

But : Redirige l'utilisateur selon s'il est connecté ou non.

```
1 <?php
2 session_start();
3 if (isset($_SESSION['token'])) {
4     header("Location: dashboard.php");
5     exit();
6 } else {
7     header("Location: login.php");
8     exit();
9 }
10 ?>
```

Listing 13.5 – `index.php`

Explication : Ce fichier joue le rôle de passerelle. S'il existe déjà un token en session, l'utilisateur est redirigé vers `dashboard.php` ; sinon, vers `login.php`.

Vue de connexion : `login.php`

But : Affiche un formulaire de connexion à l'utilisateur.

```
1 <form method="POST" action="do_login.php">
2     <input type="text" name="username" placeholder="Nom d'utilisateur" required>
3     <input type="password" name="password" placeholder="Mot de passe" required>
4     <button type="submit">Connexion</button>
5 </form>
```

Listing 13.6 – `login.php`

Explication : Cette page permet à l'utilisateur de saisir ses identifiants. Les données sont envoyées à `do_login.php` pour vérification.

Traitement de la connexion: `do_login.php`

But : Appelle l'API `/api/auth/login` pour vérifier les identifiants, puis stocke le token en session.

```
1 $response = file_get_contents($apiUrl, false, $context);
2 $data = json_decode($response, true);
3
4 if (isset($data['token'])) {
5     $_SESSION['token'] = $data['token'];
6     $_SESSION['username'] = $data['username'];
7     $_SESSION['role'] = $data['role'];
8     header("Location: dashboard.php");
9 } else {
10     echo "chec de la connexion.";
11 }
```

Listing 13.7 – `do_login.php`

Explication : Ce script vérifie les identifiants via l'API. En cas de succès, l'utilisateur est redirigé vers `dashboard.php` ; sinon, un message d'erreur est affiché.

Vue principale après connexion: `dashboard.php`

But : Affiche les infos utilisateur et propose une navigation.

```

1 <?php
2 session_start();
3 if (!isset($_SESSION['token'])) {
4     header("Location: login.php");
5     exit();
6 }
7 ?>
8
9 <p>Bienvenue, <?= $_SESSION['username'] ?> !</p>
10
11 <?php if ($_SESSION['role'] === 'ADMIN'): ?>
12     <a href="admin_users.php">Gérer les utilisateurs</a>
13 <?php endif; ?>
14
15 <a href="logout.php">Déconnexion</a>

```

Listing 13.8 – dashboard.php

Explication : Cette page est protégée par session. Elle affiche le nom de l'utilisateur connecté et, si ce dernier est administrateur, un lien vers la gestion des utilisateurs.

Vue admin de gestion des utilisateurs: `admin_users.php`

But : Permet à un administrateur de voir et modifier les permissions des utilisateurs.

```

1 <form method="POST" action="update_permissions.php">
2     <input type="hidden" name="username" value="<?= $user['username'] ?>">
3     <input type="checkbox" name="canViewTemperature" <?=
4         $user['permission']['canViewTemperature'] ? 'checked' : '' ?>>
5     <!-- autres champs -->
6     <button type="submit">Modifier</button>
</form>

```

Listing 13.9 – admin_users.php

Explication : Cette page récupère les utilisateurs via l'API `/api/admin/users/all`, affiche leurs permissions et permet à l'administrateur de les modifier.

Déconnexion: `logout.php`

But : Déconnecte l'utilisateur en supprimant la session.

```

1 <?php
2 session_start();
3 session_destroy();
4 header("Location: login.php");
5 ?>

```

Listing 13.10 – logout.php

Explication : Ce fichier détruit la session PHP et redirige l'utilisateur vers la page de connexion.